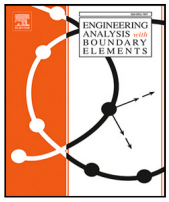
Engineering Analysis with Boundary Elements 168 (2024) 105927

Contents lists available at ScienceDirect

Engineering Analysis with Boundary Elements

journal homepage: www.elsevier.com/locate/enganabound



Gaussian smoothed particle hydrodynamics: A high-order meshfree particle method

Ni Sun [a], Ting Ye [a,*], Zehong Xia [a], Zheng Feng [a], Rusheng Wang [b,*]

[a] School of Mathematics, Jilin University, Qianjin Ave. 2699, Changchun 130012, China
[b] College of Construction Engineering, Jilin University, Xinminzhu Ave. 938, Changchun 130021, China

ARTICLE INFO

Keywords:
Meshfree particle method
Smoothed particle hydrodynamics
High-order scheme
Gaussian quadrature rule

ABSTRACT

Smoothed particle hydrodynamics (SPH) has attracted significant attention in recent decades, and exhibits special advantages in modeling complex flows with multiphysics processes and complex phenomena. Its accuracy depends heavily on the distribution of particles, and will generally be lower if the particles are distributed non-uniformly. A high-order SPH scheme is proposed in the present work for simulating both compressible and incompressible flows. It uses a Gaussian quadrature rule to perform the particle approximation of SPH by introducing Gaussian nodes. Unfortunately, the Gaussian nodes hardly overlap with SPH particles due to the Lagrangian feature, and thus we use a high-order interpolation method to obtain the corresponding physical quantities at the Gaussian nodes. The accuracy and robustness of the proposed Gaussian SPH are demonstrated by several numerical tests, including the Sod problem, Poiseuille flow, Couette flow, cavity flow, Taylor–Green vortex and dam break flow, and a convergence analysis is also conducted to evaluate the effects of particle resolution and distribution for reconstructing a given function. The simulation results for each test case are in good agreements with the available analytical, experimental or numerical results, showing that the proposed Gaussian SPH method is accurate and reliable but expensive for simulating compressible and incompressible flow problems.

1. Introduction

As a mesh-free Lagrangian method, smoothed particle hydrodynamics (SPH) has been developed considerably and received significant interest in the past decades. It was originally proposed for astrophysical problems about 40 years ago [1,2], and later modified to laboratory-scale situations like viscous flow and thermal conduction [3–5]. In 2003, it was further extended to mesoscale simulations like microfluidic flow, deriving the smoothed dissipative particle dynamics (SDPD) [6]. It is self-evident that SPH has run through the cosmo, macro and meso scales since its invention, and applied to a vast range of problems in both fluid and solid mechanics [7].

Thanks to the Lagrangian feature, SPH has special advantages over the traditional grid-based numerical methods in modeling complex fluid flows, such as free-surface flows, fluid–structure interactions and multiphase flows [8–10]. For insights into the applications of SPH, the readers may refer to some recent reviews [7,11–14]. For the same reason of the Lagrangian feature, SPH also has several inherent limitations. A prominent issue is the reduced accuracy stemming from non-uniform distribution of the particles. The Lagrangian description allows SPH particles to move freely without the geometrical topology, inevitably leading to non-uniformity in their distribution. Therefore, maintaining a uniform particle distribution in SPH is crucial for enhancing the accuracy of simulations [15,16].

Over the past decades, many innovative methodologies have emerged to improve the accuracy of SPH. An early and simple improvement involved normalizing the density to enhance the accuracy of the density approximation, particularly near the boundary or interface areas [17]. Later, Taylor series expansion was widely used to improve the approximation of functions and their derivatives, such as the corrective smoothed particle method (CSPM) [18], finite particle method (FPM) [19], and decoupled finite particle method (DFPM) [20,21]. These methods typically require calculating the inverse of a correction matrix, which leads to substantial computational costs, thereby potentially impacting the SPH efficiency. In recent years, some high-order SPH methods have been developed to further enhance accuracy with the aid of the Riemann solver, such as Godunov SPH (GSPH) [22] and weighted non-oscillatory SPH (WENO-SPH) [23]. They treat particle interactions as Riemann problems, which are resolved using exact or approximate Riemann solvers, marking a great evolution from conventional SPH approaches. In addition to algorithmic improvements,

* Corresponding authors.
E-mail addresses: yeting@jlu.edu.cn (T. Ye), wangrs@jlu.edu.cn (R. Wang).

https://doi.org/10.1016/j.enganabound.2024.105927
Received 8 April 2024; Received in revised form 17 August 2024; Accepted 18 August 2024
Available online 29 August 2024

several specialized techniques have been devised to improve the SPH accuracy. For example, the particle shifting technique helps to reduce the non-uniform distribution of particles, and has become a fundamental procedure in SPH [24–26]. The adaptive particle splitting or refinement technique, designed to prevent particle clustering and enhance resolution in areas of interest, has also been widely implemented in SPH [27–29]. Furthermore, the δ-SPH method, which introduces a density diffusive term into the continuity equation, aids in smoothing density distribution, diminishing numerical instability, and enhancing simulation accuracy, particularly at fluid interfaces [16,30,31].

In the present work, our aim is to present an idea of improving the accuracy of SPH through the integration of Gaussian quadrature. We introduce a high-order SPH approach, termed Gaussian SPH, and demonstrate its efficacy through its application to a series of benchmark problems, including the Sod shock tube, Poiseuille flow, and dam break, among others.

2. Gaussian SPH development

We solve two groups of equations that are generally considered in SPH, weakly compressible viscous flow and inviscid flow, where the former is usually allowed to approximate incompressible viscous flow. The Navier–Stokes equations read for the weakly compressible viscous flow in the Lagrangian form as,

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = -\rho\nabla\cdot\boldsymbol{v}, \tag{1}$$

$$\frac{\mathrm{d}\boldsymbol{v}}{\mathrm{d}t} = -\frac{1}{\rho}\nabla P + \nu\nabla^2\boldsymbol{v} + \boldsymbol{g}, \tag{2}$$

with an equation of state (EOS), for example,

$$P = \rho c_0^2, \tag{3}$$

where ρ is the density, $\boldsymbol{v}$ is the velocity vector, t is the time, P is the fluid pressure, ν is the kinematic viscosity, $\boldsymbol{g}$ is the gravity, and c_0 is an artificial speed of sound that is often large enough to approximate an incompressible flow.

The Euler equations read for the inviscid flow as,

$$\frac{\mathrm{d}\rho}{\mathrm{d}t} = -\rho\nabla\cdot\boldsymbol{v}, \tag{4}$$

$$\frac{\mathrm{d}\boldsymbol{v}}{\mathrm{d}t} = -\frac{1}{\rho}\nabla P, \tag{5}$$

$$\frac{\mathrm{d}e}{\mathrm{d}t} = -\frac{P}{\rho}\nabla\cdot\boldsymbol{v}, \tag{6}$$

with an EOS, for example,

$$P = \rho e(\gamma - 1), \tag{7}$$

where e is the specific internal energy, and γ is the adiabatic exponent that is often set to 1.4 for ideal gas.

2.1. Standard SPH

The SPH formulation is based on the interpolation theory, consisting of two key steps: kernel approximation and particle approximation [32]. The kernel approximation is the integral representation of a function $f(\boldsymbol{x})$, by representing $f(\boldsymbol{x})$ into an integral with a kernel function $W(\boldsymbol{x}-\boldsymbol{x}', h)$ over a domain of interest Ω (called support domain),

$$f(\boldsymbol{x}) = \int_\Omega f(\boldsymbol{x}')\delta(\boldsymbol{x}-\boldsymbol{x}')\mathrm{d}\boldsymbol{x}' = \int_\Omega f(\boldsymbol{x}')W(\boldsymbol{x}-\boldsymbol{x}', h)\mathrm{d}\boldsymbol{x}' + O\left(h^2\right), \tag{8}$$

where h is defined as the smoothing length that characterizes the size of the support domain. The particle approximation is the particle representation of $f(\boldsymbol{x})$ at a particle $\boldsymbol{x}_a$, by approximating the integral into a summation with a numerical integral quadrature,

$$f(\boldsymbol{x}_a) = \int_\Omega f(\boldsymbol{x}')W(\boldsymbol{x}_a-\boldsymbol{x}', h)\mathrm{d}\boldsymbol{x}' + O\left(h^2\right) \approx \sum_b f(\boldsymbol{x}_b)W(\boldsymbol{x}_a-\boldsymbol{x}_b, h)V_b, \tag{9}$$

where the subscript b denotes the particles inside the support domain, V_b is the volume of the particle expressed by $V_b = m_b/\rho_b$, and m_b is the particle mass. It is difficult to measure the accuracy from the integral to the summation in Eq. (9), because it depends on the uniformity of particle distribution. If the particles are uniformly distributed, the approximation has the accuracy order of $O(h^2)$,

$$f(\boldsymbol{x}_a) = \sum_b f(\boldsymbol{x}_b)W(\boldsymbol{x}_a-\boldsymbol{x}_b, h)V_b + O\left(h^2\right), \tag{10}$$

Otherwise, such an evaluation may lead to a remarkable error, which is exactly the main reason for the low accuracy of the standard SPH method [33,34]. Let $f = \rho$, $\boldsymbol{v}$, P and e in Eq. (10), ones quickly obtain the particle representations of these physical quantities, and then substituting them into the Navier–Stokes or Euler equations yields the corresponding SPH formulations. However, Eq. (10) is not directly used to express the derivatives in practice, because they are neither pairwise nor momentum-conserved [35]. We here do not focus on these drawbacks, but rather present the standard form of the first- and second-order derivatives that are widely used in SPH, for example,

$$\nabla\cdot\boldsymbol{v}_a \approx \frac{1}{\rho_a}\sum_b m_b\boldsymbol{v}_{ba}\cdot\nabla_a W_{ab}, \tag{11}$$

$$\nabla P_a \approx \rho_a\sum_b m_b\left(\frac{P_a}{\rho_a^2}+\frac{P_b}{\rho_b^2}\right)\nabla_a W_{ab}, \tag{12}$$

$$\nabla^2\boldsymbol{v}_a \approx 2\sum_b V_b\boldsymbol{v}_{ab}\frac{\boldsymbol{x}_{ab}\cdot\nabla_a W_{ab}}{|\boldsymbol{x}_{ab}|^2}, \tag{13}$$

where the subscripts are used to denote the coordinates of the corresponding particles for shortening the expressions, *e.g.*, $f_a = f(\boldsymbol{x}_a)$, $f_{ba} = f_b - f_a$, $W_{ab} = W(\boldsymbol{x}_a-\boldsymbol{x}_b, h)$, and ∇_a is the gradient with respect to $\boldsymbol{x}_a$.

Substituting Eqs. (11)–(13) into the Navier–Stokes Eqs. (1)–(3) yields the standard SPH formulation for the weakly compressible flow,

$$\frac{\mathrm{d}\rho_a}{\mathrm{d}t} = \sum_b m_b\boldsymbol{v}_{ab}\cdot\nabla_a W_{ab} \tag{14}$$

$$\frac{\mathrm{d}\boldsymbol{v}_a}{\mathrm{d}t} = -\sum_b m_b\left(\frac{P_a}{\rho_a^2}+\frac{P_b}{\rho_b^2}\right)\nabla_a W_{ab} + 2\nu\sum_b V_b\boldsymbol{v}_{ab}\frac{\boldsymbol{x}_{ab}\cdot\nabla_a W_{ab}}{|\boldsymbol{x}_{ab}|^2} + \boldsymbol{g}_a, \tag{15}$$

with the EOS

$$P_a = \rho_a c_0^2. \tag{16}$$

Similarly, substituting them into the Euler Eqs. (4)–(7), we obtain the standard SPH formulation for the inviscid flow,

$$\frac{\mathrm{d}\rho_a}{\mathrm{d}t} = \sum_b m_b\boldsymbol{v}_{ab}\cdot\nabla_a W_{ab} \tag{17}$$

$$\frac{\mathrm{d}\boldsymbol{v}_a}{\mathrm{d}t} = -\sum_b m_b\left(\frac{P_a}{\rho_a^2}+\frac{P_b}{\rho_b^2}\right)\nabla_a W_{ab}, \tag{18}$$

$$\frac{\mathrm{d}e_a}{\mathrm{d}t} = \frac{1}{2}\sum_b m_b\left(\frac{P_a}{\rho_a^2}+\frac{P_b}{\rho_b^2}\right)\boldsymbol{v}_{ab}\nabla_a W_{ab}, \tag{19}$$

with the EOS

$$P_a = \rho_a e_a(\gamma - 1). \tag{20}$$

It is found that the standard SPH formulations are mainly derived from Eq. (10), which has a low accuracy order of $O(h^2)$ for the uniform distribution of particles. If the particles are distributed randomly, the accuracy may be even lower.

2.2. Gaussian SPH

Consider the Taylor series expansion of $f(\boldsymbol{x})$ around $\boldsymbol{x}_a$ up to the sixth order,

$$f(\boldsymbol{x}) = \sum_{k=0}^{6}\frac{f^{(k)}(\boldsymbol{x}_a)}{k!}\odot(\boldsymbol{x}-\boldsymbol{x}_a)^k + O\left(h^7\right), \tag{21}$$

where $f^{(k)}(\boldsymbol{x}_a)$ and $(\boldsymbol{x}-\boldsymbol{x}_a)^k$ are two kth-order tensors, $\odot$ is used to denote a multiple dot product, for example, the product $f^{(k)}(\boldsymbol{x}_a)\odot(\boldsymbol{x}-\boldsymbol{x}_a)^k$ is the kth-order inner product, and h has the same order with $|\boldsymbol{x}-\boldsymbol{x}_a|$. It is noted from Section 2.1 that the main task in the SPH formulation is to approximate the first-order derivative ∇f and the second-order derivative $\nabla^2 f$, such as ∇P, $\nabla\cdot\boldsymbol{v}$ and $\nabla^2\boldsymbol{v}$.

2.2.1. Kernel approximation: first-order derivative

Physically, a combination of two quantities sometimes may be more important than a primitive quantity only. For example, the mass flux $\rho\boldsymbol{v}$ is compared with the velocity $\boldsymbol{v}$ only, when concerning the mass flux through a fixed volume. Another example is the combination of PV (V is the system volume) compared with the primitive variable P only, when dealing with an isothermal system. In such cases, the derivative approximation from the combinative quantity $f(\boldsymbol{x})g(\boldsymbol{x})$ is often more physical in conserving some properties, than that from the primitive quantity $f(\boldsymbol{x})$ or $g(\boldsymbol{x})$ itself. Applying the Taylor series expansion to $f(\boldsymbol{x})g(\boldsymbol{x})$ and $g(\boldsymbol{x})$ by Eq. (21) respectively, and multiplying the expansion of $g(\boldsymbol{x})$ by $f(\boldsymbol{x}_a)$, we have

$$f(\boldsymbol{x})g(\boldsymbol{x})=\sum_{k=0}^{6}\frac{1}{k!}\left(\sum_{i=0}^{k}C_k^i f^{(i)}(\boldsymbol{x}_a)g^{(k-i)}(\boldsymbol{x}_a)\right)\odot(\boldsymbol{x}-\boldsymbol{x}_a)^k+O\left(h^7\right), \tag{22}$$

$$f(\boldsymbol{x}_a)g(\boldsymbol{x})=f(\boldsymbol{x}_a)\sum_{k=0}^{6}\frac{g^{(k)}(\boldsymbol{x}_a)}{k!}\odot(\boldsymbol{x}-\boldsymbol{x}_a)^k+O\left(h^7\right), \tag{23}$$

where C_k^i is a combination number. Subtracting Eq. (23) from Eq. (22) yields,

$$(f(\boldsymbol{x})-f(\boldsymbol{x}_a))g(\boldsymbol{x})=\sum_{k=1}^{6}\frac{1}{k!}\left(\sum_{i=1}^{k}C_k^i f^{(i)}(\boldsymbol{x}_a)g^{(k-i)}(\boldsymbol{x}_a)\right)\odot(\boldsymbol{x}-\boldsymbol{x}_a)^k+O\left(h^7\right). \tag{24}$$

Based on Eq. (24), the first-order derivative of $f(\boldsymbol{x})$ is approximated by (see Appendix A)

$$\nabla f(\boldsymbol{x}_a)=f'(\boldsymbol{x}_a)=\frac{1}{2g(\boldsymbol{x}_a)a_{12}^{xx}}\left[(4+d)\boldsymbol{b}_1-\boldsymbol{b}_2\right]+O\left(h^4\right), \tag{25}$$

where

$$a_{12}^{xx}=\int_\Omega(x-x_a)^2W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{26}$$

$$\boldsymbol{b}_1=\int_\Omega[f(\boldsymbol{x})-f(\boldsymbol{x}_a)]g(\boldsymbol{x})(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{27}$$

$$\boldsymbol{b}_2=\int_\Omega[f(\boldsymbol{x})-f(\boldsymbol{x}_a)]g(\boldsymbol{x})(\boldsymbol{x}-\boldsymbol{x}_a)W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}. \tag{28}$$

with the other kernel function

$$W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)=-(\boldsymbol{x}-\boldsymbol{x}_a)\odot W'(\boldsymbol{x}-\boldsymbol{x}_a,h). \tag{29}$$

For a vector $\boldsymbol{f}(\boldsymbol{x})$, applying Eq. (25) to each component and performing a dot product with a unit second-order tensor, we have

$$\nabla\cdot\boldsymbol{f}(\boldsymbol{x}_a)=\frac{1}{2g(\boldsymbol{x}_a)a_{12}^{xx}}\left[(4+d)\dot{b}_1-\dot{b}_2\right]+O\left(h^4\right), \tag{30}$$

where

$$\dot{b}_1=\int_\Omega g(\boldsymbol{x})[\boldsymbol{f}(\boldsymbol{x})-\boldsymbol{f}(\boldsymbol{x}_a)]\odot(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{31}$$

$$\dot{b}_2=\int_\Omega g(\boldsymbol{x})[\boldsymbol{f}(\boldsymbol{x})-\boldsymbol{f}(\boldsymbol{x}_a)]\odot(\boldsymbol{x}-\boldsymbol{x}_a)W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}. \tag{32}$$

For example, if $f(\boldsymbol{x})=P(\boldsymbol{x})$ and $g(\boldsymbol{x})=1/\rho(\boldsymbol{x})$, we obtain from Eq. (25),

$$\nabla P(\boldsymbol{x}_a)=\frac{\rho(\boldsymbol{x}_a)}{2a_{12}^{xx}}\left[(4+d)\boldsymbol{b}_1^P-\boldsymbol{b}_2^P\right]+O\left(h^4\right). \tag{33}$$

with

$$\boldsymbol{b}_1^P=\int_\Omega\frac{1}{\rho(\boldsymbol{x})}[P(\boldsymbol{x})-P(\boldsymbol{x}_a)](\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{34}$$

$$\boldsymbol{b}_2^P=\int_\Omega\frac{1}{\rho(\boldsymbol{x})}[P(\boldsymbol{x})-P(\boldsymbol{x}_a)](\boldsymbol{x}-\boldsymbol{x}_a)W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}. \tag{35}$$

If $\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{v}(\boldsymbol{x})$ and $g(\boldsymbol{x})=\rho(\boldsymbol{x})$, we obtain from Eq. (30),

$$\nabla\cdot\boldsymbol{v}(\boldsymbol{x}_a)=\frac{1}{2\rho(\boldsymbol{x}_a)a_{12}^{xx}}\left[(4+d)\dot{b}_1^{\boldsymbol{v}}-\dot{b}_2^{\boldsymbol{v}}\right]+O\left(h^4\right), \tag{36}$$

with

$$\dot{b}_1^{\boldsymbol{v}}=\int_\Omega\rho(\boldsymbol{x})[\boldsymbol{v}(\boldsymbol{x})-\boldsymbol{v}(\boldsymbol{x}_a)]\odot(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{37}$$

$$\dot{b}_2^{\boldsymbol{v}}=\int_\Omega\rho(\boldsymbol{x})[\boldsymbol{v}(\boldsymbol{x})-\boldsymbol{v}(\boldsymbol{x}_a)]\odot(\boldsymbol{x}-\boldsymbol{x}_a)W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}. \tag{38}$$

2.2.2. Kernel approximation: second-order derivative

Based on Eq. (21), the second-order derivatives of $f(\boldsymbol{x})$ is approximated by (see Appendix A)

$$\nabla^2 f(\boldsymbol{x}_a)=f''_{\alpha\alpha}(\boldsymbol{x}_a)=\frac{1}{a_{12}^{xx}}\left[(4+d)c_1-c_2\right]+O\left(h^4\right), \tag{39}$$

where

$$c_1=\int_\Omega[f(\boldsymbol{x})-f(\boldsymbol{x}_a)]W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{40}$$

$$c_2=\int_\Omega[f(\boldsymbol{x})-f(\boldsymbol{x}_a)]W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{41}$$

and α is a dummy index with Einstein notation. For a vector $\boldsymbol{f}(\boldsymbol{x})$, applying Eq. (39) to each of its components yields

$$\nabla^2\boldsymbol{f}(\boldsymbol{x}_a)=\frac{1}{a_{12}^{xx}}\left[(4+d)\boldsymbol{c}_1-\boldsymbol{c}_2\right]+O\left(h^4\right), \tag{42}$$

where

$$\boldsymbol{c}_1=\int_\Omega[\boldsymbol{f}(\boldsymbol{x})-\boldsymbol{f}(\boldsymbol{x}_a)]W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{43}$$

$$\boldsymbol{c}_2=\int_\Omega[\boldsymbol{f}(\boldsymbol{x})-\boldsymbol{f}(\boldsymbol{x}_a)]W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}. \tag{44}$$

For example, $\boldsymbol{f}(\boldsymbol{x})=\boldsymbol{v}(\boldsymbol{x})$, we have,

$$\nabla^2\boldsymbol{v}(\boldsymbol{x}_a)=\frac{(4+d)\boldsymbol{c}_1^{\boldsymbol{v}}-\boldsymbol{c}_2^{\boldsymbol{v}}}{a_{12}^{xx}}+O\left(h^4\right), \tag{45}$$

with

$$\boldsymbol{c}_1^{\boldsymbol{v}}=\int_\Omega[\boldsymbol{v}(\boldsymbol{x})-\boldsymbol{v}(\boldsymbol{x}_a)]W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{46}$$

$$\boldsymbol{c}_2^{\boldsymbol{v}}=\int_\Omega[\boldsymbol{v}(\boldsymbol{x})-\boldsymbol{v}(\boldsymbol{x}_a)]W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}. \tag{47}$$

2.2.3. Particle approximation: Gaussian quadrature

It is observed from Eqs. (25) and (39) that two types of integrals are required to be calculated, to approximate a first-order or second-order derivative,

$$I_1(f)=\int_\Omega f(\boldsymbol{x})W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x},\quad I_2(f)=\int_\Omega f(\boldsymbol{x})W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{48}$$

because a_{12}^{xx} is exactly calculated for a given kernel function $W(\boldsymbol{x}-\boldsymbol{x}_a,h)$. We use the Gaussian quadrature to numerically calculate them, and this is the main reason why such a SPH method is named Gaussian SPH. Without loss of generality, we here choose a cubic B-spline kernel function that is widely used in most previous SPH methods. A 1D B-spline kernel is given by

$$W(x-x_a,h)=\frac{1}{h}\begin{cases}\frac{2}{3}-R^2+\frac{1}{2}R^3, & 0\le R<1,\\ \frac{1}{6}(2-R)^3, & 1\le R<2,\\ 0, & \text{else},\end{cases} \tag{49}$$

where $R=|x-x_a|/h$, such that a d-dimensional B-spline kernel is formed by repeating it d times, for example in a 3D case,

$$W(\boldsymbol{x}-\boldsymbol{x}_a,h)=W(x-x_a,h)W(y-y_a,h)W(z-z_a,h). \tag{50}$$

Note that the support domain is $\Omega=[x_a-2h,x_a+2h]\times[y_a-2h,y_a+2h]\times[z_a-2h,z_a+2h]$ in a 3D case, which is a cubic domain for

the convenience of performing Gaussian quadrature, different with the spherical domain in most of previous SPH methods. On one dimension, *e.g.*, x-direction, we use three-point Gaussian quadrature,

$$\int_{x_a-2h}^{x_a+2h} f(x)W(x-x_a,h)\mathrm{d}x = \sum_{i=1}^{3} A_i f(x_a+2hr_i) + O\left(h^6\right), \tag{51}$$

where A_i and r_i are quadrature coefficients and Gaussian nodes, and the kernel $W(x-x_a,h)$ is chosen as the weighted function of numerical quadrature. It is easy to obtain the quadrature coefficients and Gaussian nodes,

$$A_1 = \frac{5}{27},\ A_2 = \frac{17}{27},\ A_3 = \frac{5}{27},\quad r_1 = -\frac{3\sqrt{10}}{20},\ r_2 = 0,\ r_3 = \frac{3\sqrt{10}}{20}, \tag{52}$$

such that

$$\begin{aligned}\int_{x_a-2h}^{x_a+2h} f(x)W(x-x_a,h)\mathrm{d}x = &\left[\frac{5}{27}f\left(x_a-2h\frac{3\sqrt{10}}{20}\right)+\frac{17}{27}f(x_a)\right.\\ &\left.+\frac{5}{27}f\left(x_a+2h\frac{3\sqrt{10}}{20}\right)\right]+O\left(h^6\right).\end{aligned} \tag{53}$$

By the product formula of multiple integral [36], for example in a 3D case we have

$$I_1(f) = \sum_{i=1}^{3}\sum_{j=1}^{3}\sum_{k=1}^{3} A_iA_jA_k f(x_a+2hr_i, y_a+2hs_j, z_a+2ht_k) + O\left(h^6\right). \tag{54}$$

By transforming the multiple summations into a simple summation to simplify the expression, a more general formula in a d-dimensional case is written as

$$I_1(f) = G_1(f) + O\left(h^6\right), \tag{55}$$

and $G_1(f)$ is defined as

$$G_1(f) = \sum_{q=1}^{3^d} C_q f(\boldsymbol{x}_a + 2h\boldsymbol{r}_q), \tag{56}$$

where C_q and $\boldsymbol{r}_q$ are the quadrature coefficient and Gaussian nodes after the transformation. For convenience, $G_1(f)$ is named the first-type Gaussian quadrature operator to approximate $I_1(f)$, and similarly, $G_2(f)$ is named the second-type Gaussian quadrature operator to approximate $I_2(f)$, given by (see Appendix B)

$$G_2(f) = \sum_{q=1}^{3^d d} C'_q f(\boldsymbol{x}_a + 2h\boldsymbol{r}'_q), \tag{57}$$

where C'_q and $\boldsymbol{r}'_q$ are the quadrature coefficient and Gaussian nodes.

It is worth mentioning in Gaussian quadrature that a weighted function $W(x)$ is chosen only if it satisfies the following conditions,

$$W(x) \geq 0, \quad \int_\Omega W(x)\mathrm{d}x = 1. \tag{58}$$

This means the SPH kernel functions should be chosen properly to perform Gaussian quadrature. For this purpose, we multiply such special odd or even functions, $W(\boldsymbol{x}-\boldsymbol{x}_a,h)$, $W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)$, $(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)$, and $(\boldsymbol{x}-\boldsymbol{x}_a)W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)$, when carrying out kernel approximation in Appendix A.

2.2.4. Particle approximation: interpolation at Gaussian nodes

The Gaussian quadrature significantly improves the accuracy of numerically calculating the integrations in Eq. (48), compared with the method used in the standard SPH, but requires the function values at the Gaussian nodes. It is found in Eqs. (56) and (57) that there are $3^d(1+d)$ Gaussian nodes, including $3^{d-1}d$ repeated ones. Hence, to approximate a first- or second-order derivative at a point $\boldsymbol{x}_a$, we need to evaluate $f(\boldsymbol{x})$ at $3^{d-1}(3+2d)$ Gaussian nodes, *i.e.*, 5, 21 and 81 non-repeated nodes in 1D, 2D and 3D cases, respectively. For example, in a 1D case the six Gaussian nodes are

$$r_1 = -\frac{3\sqrt{10}}{20},\ r_2 = 0,\ r_3 = \frac{3\sqrt{10}}{20},\ r'_1 = -\frac{\sqrt{6}}{4},\ r'_2 = 0,\ r'_3 = \frac{\sqrt{6}}{4}, \tag{59}$$

with a repeated node of $r_2 = r'_2 = 0$. These nodes hardly coincide with the SPH particles so that the interpolation process is required.

There are four interpolating approaches to obtain the function value $f(\boldsymbol{x}_q)$ at a Gaussian node $\boldsymbol{x}_q$. The first one (abbreviated as Interp I) is to truncate the right term of Eq. (A.25) up to the second-order derivative,

$$f(\boldsymbol{x}_q) = \int_\Omega f(\boldsymbol{x})W(\boldsymbol{x}-\boldsymbol{x}_q,h)\mathrm{d}\boldsymbol{x} + O\left(h^2\right) \approx \sum_p V_p f_p W_{pq}, \tag{60}$$

where the subscript p denotes the particles inside the support domain of $\boldsymbol{x}_q$. As shown in Eq. (10), this evaluation has low accuracy with $O(h^2)$ when the particles are uniformly distributed. If the particles are distributed randomly, a large error may be produced.

The second approach (Interp II) is to combine Eqs. (A.25) and (A.26), and truncating to the fourth-order derivatives yields

$$\begin{aligned}f(\boldsymbol{x}_q) &= \frac{1}{2}\int_\Omega f(\boldsymbol{x})\left[(2+d)W(\boldsymbol{x}-\boldsymbol{x}_q,h) - W_1(\boldsymbol{x}-\boldsymbol{x}_q,h)\right]\mathrm{d}\boldsymbol{x} + O\left(h^4\right)\\ &\approx \frac{1}{2}\sum_p V_p f_p\left[(2+d)W_{pq} - W_{1,pq}\right].\end{aligned} \tag{61}$$

There exists a similar issue about the approximation of integral by a summation, but if the particles are uniformly distributed, we have

$$\begin{aligned}\int_\Omega f(\boldsymbol{x})\left[(2+d)W(\boldsymbol{x}-\boldsymbol{x}_q,h) - W_1(\boldsymbol{x}-\boldsymbol{x}_q,h)\right]\mathrm{d}\boldsymbol{x} = \sum_p V_p f_p\left[(2+d)W_{pq}\right.\\ \left.-W_{1,pq}\right] + O\left(h^4\right).\end{aligned} \tag{62}$$

The second approach has a higher accuracy than the first one, but spends similar computational costs.

The third approach (Interp III) is to choose the particle $\boldsymbol{x}_p$ that is the closest to $\boldsymbol{x}_q$, and then to give the Taylor series expansion of $f(\boldsymbol{x}_q)$ around $\boldsymbol{x}_p$ with the first-order derivative,

$$f(\boldsymbol{x}_q) = f(\boldsymbol{x}_p) + f'(\boldsymbol{x}_p)\odot(\boldsymbol{x}_q-\boldsymbol{x}_p) + O\left(|\boldsymbol{x}_p-\boldsymbol{x}_q|^2\right). \tag{63}$$

The accuracy of this evaluation depends on $|\boldsymbol{x}_p-\boldsymbol{x}_q|$. In general, $|\boldsymbol{x}_p-\boldsymbol{x}_q|$ is quite smaller than h so that the accuracy is often acceptable. However, it still needs the unknown value of $f'(\boldsymbol{x}_p)$ that is to be determined in SPH [23]. For example, we use Eq. (25) with $g(\boldsymbol{x}) = 1$ to evaluate it,

$$f'(\boldsymbol{x}_p) \approx \frac{1}{2a_{12}^{xx}}\sum_c V_c f_{cp}\left[(4+d)W_{cp} - W_{1,cp}\right]\boldsymbol{x}_{cp}, \tag{64}$$

where the subscript c denotes the particles inside the support domain of the particle $\boldsymbol{x}_p$, and thus the third approach gives

$$f(\boldsymbol{x}_q) \approx f(\boldsymbol{x}_p) + \frac{1}{2a_{12}^{xx}}\sum_c V_c f_{cp}\left[(4+d)W_{cp} - W_{1,cp}\right]\boldsymbol{x}_{cp}\odot\boldsymbol{x}_{qp}. \tag{65}$$

A similar issue about the approximation of integral by a summation is also present when the particles are distributed non-uniformly. Each of the above approaches given by Eqs. (60), (61) and (65) includes one summation over the support domain to approximate $f(\boldsymbol{x}_q)$, such that there are totally $3^{d-1}(3+2d)$ summations in a d-dimensional case to approximate the first- or second-order derivatives. Compared with the standard SPH that only requires one summation, the above three approaches significantly increase the computational costs.

The fourth approach (Interp IV) is to avoid the summation by performing the interpolation on an element that is formed by the k-nearest neighbors of $\boldsymbol{x}_q$. Taking the linear interpolation for example, we need find $d+1$ nearest neighbors of $\boldsymbol{x}_q$ for the d-dimensional case,

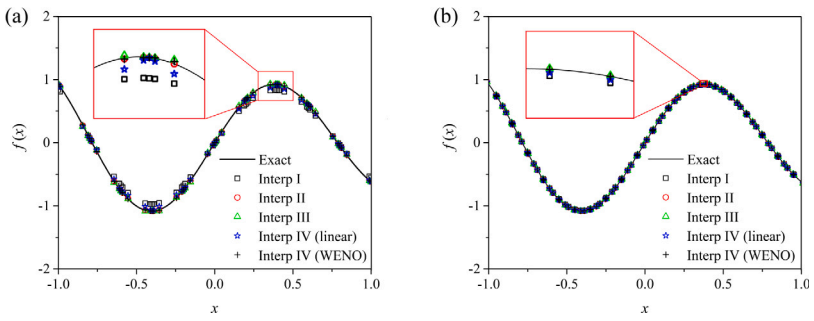


Fig. 1. Comparisons among the four interpolating approaches for calculating $f(x)$ at the Gaussian nodes, under (a) $\Delta x = 0.2$ and (b) $\Delta x = 0.05$.

denoted as $\boldsymbol{x}_{q,1}$, $\boldsymbol{x}_{q,2}$, $\cdots$, $\boldsymbol{x}_{q,d+1}$, respectively. The function value is thus evaluated by

$$f(\boldsymbol{x}_q) \approx \varphi_1(\boldsymbol{x}) f(\boldsymbol{x}_{q,1}) + \varphi_2(\boldsymbol{x}) f(\boldsymbol{x}_{q,2}) + \cdots + \varphi_{d+1}(\boldsymbol{x}) f(\boldsymbol{x}_{q,d+1}), \tag{66}$$

where $\varphi_i(\boldsymbol{x})$ is the basis function with $i = 1, 2, \ldots, d+1$. If the k-nearest neighbors are close to $\boldsymbol{x}_q$, this evaluation can provide a satisfactory approximation, and the computational costs caused by the summation are converted to that spent by searching the k-nearest neighbors. In addition, it is found that the kernel approximation in Eqs. (25) and (39) and the Gaussian quadrature in Eqs. (55) and (B.6) have the accuracy of at least $O(h^4)$, if the function values at Gaussian nodes are exactly correct. Hence, the accuracy of Gaussian SPH is mainly determined by the interpolating accuracy at Gaussian nodes. Following this idea, we can use more accurate interpolating methods to replace the linear interpolation, such as the fifth-order polynomial interpolation, or the fifth-order weighted non-oscillatory (WENO) scheme, and especially, the WENO scheme performs well even if the function is discontinuous. More details about the WENO scheme can be found in the work of Shu [37].

To compare these four approaches, we reconstruct a simple function,

$$f(x) = \mathrm{e}^{-0.2x} \sin(4x), \quad x \in [-1, 1]. \tag{67}$$

The particles are equidistantly generated with the step of Δx and the smoothing length is fixed to $h = \Delta x$. Fig. 1 shows the interpolation of $f(x)$ at the Gaussian nodes of all the particles, under $\Delta x = 0.2$ and 0.05, respectively. At $\Delta x = 0.2$, Interp I and IV (linear) have large errors, while Interp II, III and IV (WENO) are more accurate. At $\Delta x = 0.05$, all the interpolating approaches can generate satisfactory function values. Table 1 lists the L_1 errors of each approach under the six particle resolutions of $\Delta x = 0.2$, 0.1, 0.05, 0.025, 0.0125 and 0.00625, respectively. The L_1 error is defined by

$$L_1 = \frac{1}{N} \sum_{i=1}^{N} \left| f_i^n - f_i^e \right|, \tag{68}$$

where f_i^n and f_i^e are the numerical and exact values of a function $f(x)$ at the point x_i, and N is the total number of particles. For example, at $\Delta x = 0.05$, the L_1 error is about 10^{-5} in the Interp II and IV (WENO) approaches, 10^{-4} in the Interp III approach, and 10^{-3} in the Interp I and IV (linear) approaches. Table 2 lists the computational costs of each approach under these six particle resolutions. The Interp I, II, III and Interp IV (linear) spend the similar computational costs, whereas the Interp IV (WENO) is about 50 times more expensive. For instance, at $\Delta x = 0.00625$, the computational cost is about 3.1 s in the previous four approaches, but 185.7 s in the last one. Considering both the accuracy and computational cost, we suggest the Interp II approach for a large-scale problem, and the Interp IV (WENO) approach for a small-scale problem.

2.3. Gaussian SPH for Navier–Stokes equations

Applying Eqs. (56) and (57) to Eqs. (33), (36) and (45), we directly have

$$\rho_a \nabla \cdot \boldsymbol{v}_a \approx \frac{1}{2a_{12}^{xx}} \left[(4+d) G_1(\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) - G_2(\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) \right], \tag{69}$$

$$\frac{\nabla P_a}{\rho_a} \approx \frac{1}{2a_{12}^{xx}} \left[(4+d) G_1(\bar{P}/\rho \bar{\boldsymbol{x}}) - G_2(\bar{P}/\rho \bar{\boldsymbol{x}}) \right], \tag{70}$$

$$\nabla^2 \boldsymbol{v}_a \approx \frac{1}{a_{12}^{xx}} \left[(4+d) G_1(\bar{\boldsymbol{v}}) - G_2(\bar{\boldsymbol{v}}) \right], \tag{71}$$

where the overline of a function $f(\boldsymbol{x})$ is defined as $\bar{f}(\boldsymbol{x}) = f(\boldsymbol{x}) - f(\boldsymbol{x}_a)$, and a_{12}^{xx} is exactly calculated when the weighted function is given. For example, if the B-spline weighted function in Eq. (49) is used, we have

$$a_{12}^{xx} = \frac{h^2}{3}. \tag{72}$$

Substituting Eqs. (69)–(71) into the Navier–Stokes Eqs. (1)–(3) yields the formulation of Gaussian SPH for the weakly compressible flow,

$$\frac{\mathrm{d}\rho_a}{\mathrm{d}t} = -\frac{1}{2a_{12}^{xx}} \left[(4+d) G_1(\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) - G_2(\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) \right], \tag{73}$$

$$\begin{aligned} \frac{\mathrm{d}\boldsymbol{v}_a}{\mathrm{d}t} = &-\frac{1}{2a_{12}^{xx}} \left[(4+d) G_1(\bar{P}/\rho \bar{\boldsymbol{x}}) - G_2(\bar{P}/\rho \bar{\boldsymbol{x}}) \right] \\ &+ \frac{\nu}{a_{12}^{xx}} \left[(4+d) G_1(\bar{\boldsymbol{v}}) - G_2(\bar{\boldsymbol{v}}) \right], \end{aligned} \tag{74}$$

with the EOS given by Eq. (16).

Moreover, if $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{v}(\boldsymbol{x})$ and $g(\boldsymbol{x}) = P(\boldsymbol{x})/\rho(\boldsymbol{x})$ in Eq. (30), we have a new approximation of $\nabla \cdot \boldsymbol{v}_a$ by applying Eqs. (56) and (57) again,

$$\frac{P_a}{\rho_a} \nabla \cdot \boldsymbol{v}_a \approx \frac{1}{2a_{12}^{xx}} \left[(4+d) G_1(P/\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) - G_2(P/\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) \right]. \tag{75}$$

Similarly, substituting Eqs. (69), (70) and (75) into the Euler Eqs. (4)–(7), we obtain the formulation of Gaussian SPH for the inviscid flow,

$$\frac{\mathrm{d}\rho_a}{\mathrm{d}t} = -\frac{1}{2a_{12}^{xx}} \left[(4+d) G_1(\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) - G_2(\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) \right], \tag{76}$$

$$\frac{\mathrm{d}\boldsymbol{v}_a}{\mathrm{d}t} = -\frac{1}{2a_{12}^{xx}} \left[(4+d) G_1(\bar{P}/\rho \bar{\boldsymbol{x}}) - G_2(\bar{P}/\rho \bar{\boldsymbol{x}}) \right], \tag{77}$$

$$\frac{\mathrm{d}e_a}{\mathrm{d}t} = -\frac{1}{2a_{12}^{xx}} \left[(4+d) G_1(P/\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) - G_2(P/\rho \bar{\boldsymbol{v}} \odot \bar{\boldsymbol{x}}) \right], \tag{78}$$

with the EOS given by Eq. (20).

In order to evolve the particle position, one more equation is required for both the weakly compressible or inviscid flow,

$$\frac{\mathrm{d}\boldsymbol{x}_a}{\mathrm{d}t} = \boldsymbol{v}_a. \tag{79}$$

2.4. Runge–Kutta methods for time discretization

After the spatial discretization, the Navier–Stokes equations become ordinary differential equations, written as

$$\boldsymbol{U}_t = \boldsymbol{L}(\boldsymbol{U}). \tag{80}$$

For example, Eqs. (73) and (74) for the weakly compressible equations are written as,

$$\boldsymbol{U} = \begin{pmatrix} \rho_a \\ \boldsymbol{v}_a \end{pmatrix},$$

Table 1
The L_1 errors of $f(x)$ with the four interpolating approaches under the different Δx but $h = \Delta x$.

Δx	Interp I	Interp II	Interp III	Interp IV (Linear)	Interp IV (WENO)
0.2	6.3614×10^{-2}	3.4440×10^{-3}	6.8934×10^{-3}	1.8690×10^{-2}	2.1978×10^{-3}
0.1	1.6077×10^{-2}	4.1906×10^{-4}	1.2748×10^{-3}	4.3908×10^{-3}	1.8600×10^{-4}
0.05	3.9802×10^{-3}	5.1956×10^{-5}	2.7344×10^{-4}	1.0703×10^{-3}	2.4610×10^{-5}
0.025	9.8982×10^{-4}	6.4764×10^{-6}	6.4494×10^{-5}	2.6509×10^{-4}	3.1509×10^{-6}
0.0125	2.4686×10^{-4}	8.0886×10^{-7}	1.5835×10^{-5}	6.6071×10^{-5}	3.9422×10^{-7}
0.00625	6.1658×10^{-5}	1.0109×10^{-7}	3.9378×10^{-6}	1.6495×10^{-5}	4.8981×10^{-8}

Table 2
The computational costs (s) of $f(x)$ with the four interpolating approaches under the different Δx but $h = \Delta x$.

Δx	Interp I	Interp II	Interp III	Interp IV (Linear)	Interp IV (WENO)
0.2	0.1206	0.1200	0.1277	0.1240	6.4665
0.1	0.2169	0.2139	0.2249	0.2168	12.325
0.05	0.4080	0.4071	0.4206	0.4131	23.867
0.025	0.7965	0.7933	0.8116	0.7964	47.120
0.0125	1.5642	1.5685	1.5982	1.5369	93.166
0.00625	3.1218	3.1197	3.1847	3.1227	185.71

$$L(\boldsymbol{U}) = \begin{pmatrix} -\frac{1}{2a_{12}^{xx}}\left[(4+d)G_1(\rho\bar{\boldsymbol{v}}\odot\bar{\boldsymbol{x}}) - G_2(\rho\bar{\boldsymbol{v}}\odot\bar{\boldsymbol{x}})\right] \\ -\frac{1}{2a_{12}^{xx}}\left[(4+d)G_1(\bar{P}/\rho\bar{\boldsymbol{x}}) - G_2(\bar{P}/\rho\bar{\boldsymbol{x}})\right] + \frac{\nu}{a_{12}^{xx}}\left[(4+d)G_1(\bar{\boldsymbol{v}}) - G_2(\bar{\boldsymbol{v}})\right] \end{pmatrix}, \tag{81}$$

and Eqs. (76)–(78) for the inviscid flow are written as

$$\boldsymbol{U} = \begin{pmatrix} \rho_a \\ \boldsymbol{v}_a \\ e_a \end{pmatrix}, \quad L(\boldsymbol{U}) = \begin{pmatrix} -\frac{1}{2a_{12}^{xx}}\left[(4+d)G_1(\rho\bar{\boldsymbol{v}}\odot\bar{\boldsymbol{x}}) - G_2(\rho\bar{\boldsymbol{v}}\odot\bar{\boldsymbol{x}})\right] \\ -\frac{1}{2a_{12}^{xx}}\left[(4+d)G_1(\bar{P}/\rho\bar{\boldsymbol{x}}) - G_2(\bar{P}/\rho\bar{\boldsymbol{x}})\right] \\ -\frac{1}{2a_{12}^{xx}}\left[(4+d)G_1(P/\rho\bar{\boldsymbol{v}}\odot\bar{\boldsymbol{x}}) - G_2(P/\rho\bar{\boldsymbol{v}}\odot\bar{\boldsymbol{x}})\right] \end{pmatrix}. \tag{82}$$

To match the accuracy of the spatial discretization, we use the third-order Runge–Kutta method on temporal discretization,

$$\boldsymbol{U}^{(1)} = \boldsymbol{U}^n + \Delta t L(\boldsymbol{U}^n), \tag{83}$$

$$\boldsymbol{U}^{(2)} = \frac{3}{4}\boldsymbol{U}^n + \frac{1}{4}\boldsymbol{U}^{(1)} + \frac{1}{4}\Delta t L\left(\boldsymbol{U}^{(1)}\right), \tag{84}$$

$$\boldsymbol{U}^{n+1} = \frac{1}{3}\boldsymbol{U}^n + \frac{2}{3}\boldsymbol{U}^{(2)} + \frac{2}{3}\Delta t L\left(\boldsymbol{U}^{(2)}\right), \tag{85}$$

where Δt is the time step. In the implementation, Eq. (79) is decoupled for saving computational costs, and first solved by

$$\boldsymbol{x}_a^{n+1} = \boldsymbol{x}_a^n + \Delta t \boldsymbol{v}_a^n, \tag{86}$$

to evolve the particle positions. Then, Eq. (81) or (82) is solved by the third-order Runge–Kutta method to update the density, velocity and energy. The simulations show that the decoupling strategy has almost no effect on the accuracy of Gaussian SPH. The computational costs are mostly spent on the calculation of $L(\boldsymbol{U}^n)$, $L(\boldsymbol{U}^{(1)})$ and $L(\boldsymbol{U}^{(2)})$, in which the interpolation of Gaussian nodes occupies a dominant part. It is highlighted again that there are $3^{d-1}(3+2d)$ Gaussian nodes for each SPH particle, and the function values at each Gaussian node need to be required.

3. Results and discussion

3.1. Reconstruction of derivatives

To investigate the numerical convergence of the Gaussian SPH, we reconstruct the first- and second-order derivatives of $f(x)$ in Eq. (67). Fig. 2(a–b) shows the reconstruction results under $\Delta x = 0.05$ with the Interp IV (linear and WENO), and the smoothing length is set as $h = \Delta x$. The first-order derivative is well approximated with both the linear and WENO Interp IV methods, but the second-order derivative is accurately approximated by the WENO Interp IV method only. Table 3 lists the L_1 errors of $f'(x)$ and $f''(x)$, under the six particle resolutions, $\Delta x = 0.2$, 0.1, 0.05, 0.025, 0.0125, 0.00625. It is clear that the L_1 errors of $f'(x)$ and $f''(x)$ decrease with increasing the particle resolution, except for the second-order derivative by using the linear Interp IV approach. This is because the linear Interp IV approach cannot reconstruct $f(x)$ accurately enough. Considering the B-spline weighted function in Eq. (49), suppose $\tilde{f}(x) = f(x) + \varepsilon(x)$ with $|\varepsilon(x)| \le \varepsilon_m$, we obtain from Eq. (39),

$$\left|\nabla^2\varepsilon(x_a)\right| \approx \left|\frac{(4+d)c_1 - c_2}{h^2/3}\right| \le \frac{36\varepsilon_m}{h^2}. \tag{87}$$

This implies that the error $\varepsilon(x)$ of $f(x)$ is amplified by $36/h^2$, when calculating its second-order derivative. For example, at $\Delta x = 0.00625$, the linear Interp IV method has the error of 1.6495×10^{-5} given in Table 1, which leads to the error limit of $f''(x)$ about 10. Although the error of $f(x)$ is small, the error of $f''(x)$ is very large, because the smoothing length h is quite small. However, the WENO Interp IV method has the error of 4.8981×10^{-8}, causing the error limit of $f''(x)$ about 0.03. This tells us a fact that the smoothing length h should be small to reduce the error of kernel approximation, but it cannot be so small that the second-order derivative is approximated badly. Moreover, the Interp II approach also has this issue in the case of the non-uniform particle distribution, where the error of function value is not as small as that in the case of the uniform particle distribution.

Then, we examine the effect of the smoothing length h. Intuitively, the shorter the smoothing length (the smaller h), the more accurate the kernel approximation; the higher the particle resolution (the smaller Δx), the more accurate the particle approximation. However, there should be a balance between h and Δx. If $\Delta x/h$ is enough large, there are almost no particles in the support domain, and thus the particle approximation is not accurate. On the contrary, if $\Delta x/h$ is too small, the smoothing length is quite large, such that the kernel approximation has low accuracy. Quinlan et al. [33] gave an estimation of $f'(x)$ for the standard SPH,

$$f'_a - \sum_b V_b f_b W'_{ab} = O\left(h^2\right) + O\left(\frac{\Delta x}{h}\right)^{\beta+2}, \tag{88}$$

where β is the boundary smoothness of the kernel function. Hence, the smoothing length should have an optimal value. We here set $h = \alpha\Delta x$, and vary $\alpha = 0.5$, 0.7, 1.0, 1.2 and 1.5, respectively. For each α we consider the L_1 error under the above six particle resolutions. Fig. 2(c–d) shows the L_1 error of $f'(x)$ and $f''(x)$ at the different α with the WENO Interp IV approach. The convergence rate of $f'(x)$ is observed to be between 2 and 3, but between 1 and 2 for $f''(x)$. These are not the same as the theoretical orders of $O(h^4)$, due to the limitation of the interpolation accuracy. At each fixed α, the L_1 errors of $f'(x)$ and $f''(x)$ almost decrease as the number N of particles is increased or Δx is decreased. However, the errors may not vanish as Δx goes to zero, but rather becomes dominated by the residual term of $\Delta x/h$ if noting Eq. (88). This is also observed when calculating $f''(x)$ by the Interp II approach. Moreover, at each fixed Δx, the L_1 errors of $f'(x)$ and $f''(x)$ are not monotonic with respect to α, but have a fluctuation behavior. For example, at $\Delta x = 0.1$ ($\mathrm{Log}(N) \approx 1.25$) in Fig. 2(c), the L_1 error of $f'(x)$ decreases first from $\alpha = 0.5$, 0.7 to 1.0, and then increases from $\alpha = 1.0$, 1.2 to 1.5. The smoothing length achieves an optimal value at $\alpha = 1.0$, having the minimum errors of $f'(x)$ and $f''(x)$. Such a

Table 3
The L_1 errors of $f'(x)$ and $f''(x)$ with the uniform particle distribution, under the different Δx and $h = \Delta x$ by the linear and WENO Interp IV approaches, respectively.

Δx	1st derivative $f'(x)$		2nd derivative $f''(x)$	
	Interp IV (linear)	Interp IV (WENO)	Interp IV (linear)	Interp IV (WENO)
0.2	1.2597×10^{-1}	3.9591×10^{-2}	4.0554×10^{-1}	2.5632×10^{-1}
0.1	4.4061×10^{-2}	6.3311×10^{-3}	1.6217×10^{-1}	6.6612×10^{-2}
0.05	1.1798×10^{-2}	1.7787×10^{-3}	3.1753×10^{-1}	2.0607×10^{-2}
0.025	3.0015×10^{-3}	4.5706×10^{-4}	3.5674×10^{-1}	9.4927×10^{-3}
0.0125	7.5377×10^{-4}	1.1402×10^{-4}	3.6625×10^{-1}	4.8179×10^{-3}
0.00625	1.8864×10^{-4}	2.8302×10^{-5}	3.6851×10^{-1}	2.4189×10^{-3}

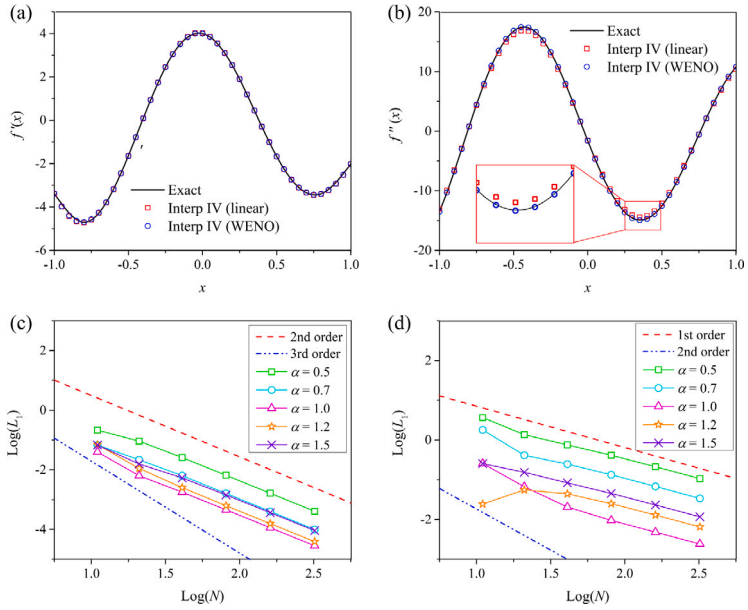


Fig. 2. Reconstructions of derivatives with the linear and WENO Interp IV approaches under the uniform particle distribution with $\Delta x = 0.05$ and $h = \Delta x$, (a) the first-order derivative $f'(x)$, (b) the second-order derivative $f''(x)$, (c) the convergence rate of $f'(x)$ and (d) the convergence rate of $f''(x)$ under different smoothing lengths $h = \alpha\Delta x$.

fluctuation behavior is a common phenomenon, and Quinlan et al. [33] suggested that the numerous local minima of the error corresponds to the points where it changes sign.

Now, we examine the effect of non-uniform particle distributions. The non-uniform particle distribution is generated by modifying a uniform distribution x with a random perturbation [23],

$$x' = x + \xi(R_n - 0.5)\Delta x, \tag{89}$$

where R_n is a random number whose value is between 0 and 1, ξ is a constant, $\xi = 0$ corresponds to a uniform particle distribution, $\xi > 0$ leads to a non-uniform particle distribution, and here we choose $\xi = 0.5$. Fig. 3 shows the reconstructions and convergence rates of $f'(x)$ and $f''(x)$ under the non-uniform particle distributions. The results do not get much worse, compared with the uniform particle distribution. The convergence rate is not changed, still between 2 and 3 for $f'(x)$, and 1 and 2 for $f''(x)$, while the L_1 errors of $f'(x)$ and $f''(x)$ are larger than those under the uniform distribution. For example, it is 2.0607×10^{-2} for $f''(x)$ at $\Delta x = 0.05$ and $h = \Delta x$ under the uniform distribution, but 1.3141×10^{-1} under the non-uniform distribution. It has been known that the accuracy of the SPH method is influenced by many factors, such as the smoothing error and discretization error, and thus in practice it is difficult to increase the formal order. Although the present Gaussian SPH has an accuracy order not as high as the mesh-based methods, it is as accurate as some high-order SPH methods. For example, Wang et al. [23] developed a WENO-SPH method that has the convergence rate of between 2 and 3 for $f'(x)$, and between 1 and 2 for $f''(x)$.

It is found from the above analysis that the second-order derivative is approximated with a low convergence rate of between 1 and 2. The main reason is that the function itself is not approximated accurately enough, although its L_1 error has already been about 10^{-8}. It may be difficult for a particle-based method to further improve the approximation accuracy from 10^{-8}. One of the efficient ways is to decrease the value of $36\varepsilon_m/h^2$ by enlarging h in Eq. (87), *e.g.*, setting $h \approx \sqrt{\Delta x}$ here. In such a way, the accuracy of the kernel approximation will be reduced, and meanwhile, the particle number in the supporting domain will be increased, largely increasing the computational costs for the standard SPH. Here, we increase the convergence rate of $f''(x)$ by enlarging h, at the cost of reducing the accuracy of the kernel approximation that may depress the convergence rate of the first-order derivative. Table 4 lists the comparison of the L_1 error of $f'(x)$ and $f''(x)$ between $h = \Delta x$ and $h = \sqrt{\Delta x}$, under the non-uniform particle distribution with $\xi = 0.5$. The error of $f'(x)$ is enlarged by about three times as h is changed from Δx to $\sqrt{\Delta x}$, while the error of $f''(x)$ is reduced by about 20 times. Fig. 3(e and f) show the reconstruction results and the convergence rates, in which both $f'(x)$ and $f''(x)$ are well approximated and converged with the order between 2 and 3.

To sum up, the present Gaussian SPH can reconstruct a continuous function with the error of about 10^{-8}, and its first- and second-order derivatives with the error of about 10^{-4}, under the non-uniform particle distribution with $\Delta x = 0.00625$. Both the first- and second-order derivatives have the convergence rates of between 2 and 3.

3.2. Inviscid flow: Sod problem

A shock tube problem considered by Sod [38] is simulated as the benchmark of inviscid flow. It is governed by Eqs. (4)–(7), and the

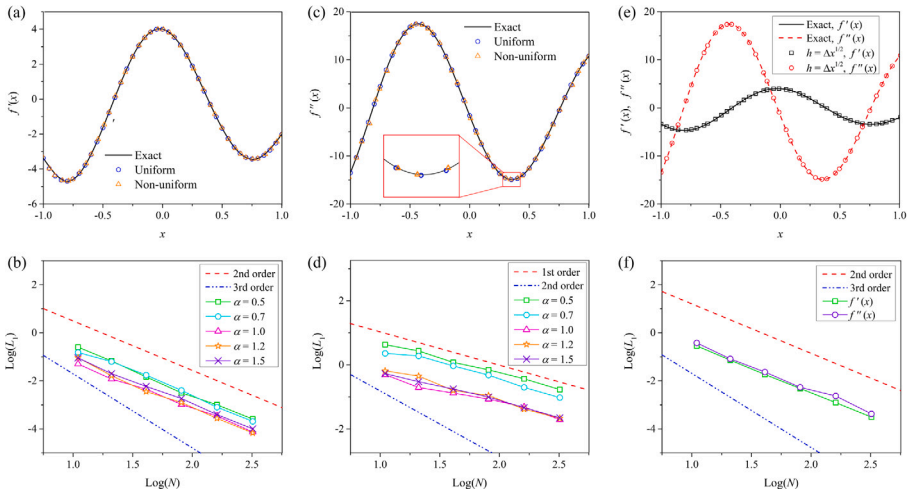


Fig. 3. Reconstructions of derivatives with the WENO Interp IV approach under the non-uniform particle distribution with a perturbation constant of $\xi = 0.5$ and $\Delta x = 0.05$, (a) the first-order derivative $f'(x)$ under $h = \Delta x$, (b) the convergence rate of $f'(x)$ under $h = \alpha\Delta x$, (c) the second-order derivative $f''(x)$ under $h = \Delta x$, (d) the convergence rate of $f''(x)$ under $h = \alpha\Delta x$, (e) the first- and second-order derivatives under $h = \sqrt{\Delta x}$, and (f) the convergence rates of $f'(x)$ and $f''(x)$ under $h = \sqrt{\Delta x}$.

Table 4
The L_1 errors of $f'(x)$ and $f''(x)$ with the non-uniform particle distribution of $\xi = 0.5$, under $h = \Delta x$ and $h = \sqrt{\Delta x}$, respectively.

Δx	1st derivative		2nd derivative	
	$h = \Delta x$	$h = \sqrt{\Delta x}$	$h = \Delta x$	$h = \sqrt{\Delta x}$
0.2	5.0759×10^{-2}	2.8575×10^{-1}	4.9494×10^{-1}	3.8009×10^{-1}
0.1	1.1853×10^{-2}	7.2610×10^{-2}	1.9589×10^{-1}	8.2283×10^{-2}
0.05	4.5294×10^{-3}	1.8508×10^{-2}	1.3141×10^{-1}	2.3356×10^{-2}
0.025	1.0419×10^{-3}	4.8179×10^{-3}	8.5409×10^{-2}	5.3614×10^{-3}
0.0125	3.4367×10^{-4}	1.2407×10^{-4}	4.8034×10^{-2}	2.3467×10^{-3}
0.00625	7.4576×10^{-5}	3.0977×10^{-4}	1.9675×10^{-2}	4.2287×10^{-4}

initial condition is

$$(P, \rho, u) = \begin{cases} (1, 1, 0), & -0.5 \le x < 0, \\ (0.1, 0.125, 0), & 0 \le x \le 0.5. \end{cases} \tag{90}$$

This initial condition leads to a shockwave that propagates at the front, followed by a contact wave, while a rarefaction travels in the opposite direction. In a simulation, 400 particles are uniformly arranged in the whole domain $[-0.5, 0.5]$ at the initial time, *i.e.* $\Delta x = 0.0025$. Because there are no second-order derivatives for the Euler equations, we here set $h = \Delta x$. The time step is set to be $\Delta t = 5.0 \times 10^{-4}$, and the final physical time is $t = 0.15$. The function values at the Gaussian nodes are interpolated by the WENO Interp IV method.

Fig. 4 shows the present numerical results which are compared with the exact solution and those obtained by the WENO-SPH [23] and the Godunov SPH [22]. It is observed that both the density and specific internal energy are somewhat dissipative, but still in good agreement with the exact solution. The dissipation is attributed to the artificial viscosity, which is necessary for the present Gaussian SPH to avoid spurious oscillation. We here adopt a simple form,

$$\hat{\boldsymbol{U}}_i^n = \boldsymbol{U}_i^n + \frac{1}{2}\theta\eta\left(\boldsymbol{U}_{i-1}^n - 2\boldsymbol{U}_i^n + \boldsymbol{U}_{i+1}^n\right), \tag{91}$$

where $\boldsymbol{U}$ is the variable vector defined in Eq. (82), η is the coefficient of the artificial viscosity and set to be 0.5 here, and θ is a parameter to indicate the position of shock,

$$\theta = \left| \frac{|\rho_{i+1} - \rho_i| - |\rho_i - \rho_{i-1}|}{|\rho_{i+1} - \rho_i| + |\rho_i - \rho_{i-1}|} \right|. \tag{92}$$

It is clear that θ is small if ρ is varied smoothly, or it becomes large if ρ is discontinuous. Because of this artificial viscosity, the Gaussian SPH looks more accurate than the WENO-SPH and Godunov SPH, without any spurious oscillation. Hence, it has the same accuracy as the WENO-SPH and Godunov SPH that are inherently high-order methods based on the Riemann solver. To reduce the dissipation, we use 1000 particles to discretize the interval, and the simulation results become more accurate, as shown in Fig. 4(c-d). In the future, we will apply the upwind scheme to the Gaussian SPH for removing the artificial viscosity, and also attempt 2D and 3D problems. In the present work, we do not pay more effort to the inviscid flows, but rather to the weakly compressible flow.

3.3. Weakly compressible flow: Poiseuille flow

An unsteady Poiseuille flow is simulated as the benchmark of weakly compressible flow. The simulation domain is given as $L \times D = 0.02 \times 0.01$ m^2 and the fluid density is $\rho = 10^3$ kg/m^3. The kinematic viscosity and the gravity are adjusted with the Reynolds number that is defined as $\mathrm{Re} = v_m D/\nu$, where v_m is the maximum flow velocity and fixed to 0.02 m/s. To set Re = 20, the kinematic viscosity is required to be $\nu = 10^{-5}$ m^2/s, and the gravity is $\boldsymbol{g} = (1.6 \times 10^{-2}, 0)$ m/s^2. To set Re = 200, $\nu = 10^{-6}$ m^2/2 and $\boldsymbol{g} = (1.6 \times 10^{-3}, 0)$ m/s^2, and to set Re = 2000, $\nu = 10^{-7}$ m^2/s and $\boldsymbol{g} = (1.6 \times 10^{-4}, 0)$ m/s^2. There are 800 particles with $\Delta x = 5 \times 10^{-4}$ m, the time step is $\Delta t = 10^{-3}$ s, and the smoothing length is $h = \Delta x$. The total physical time is set to be $T = 10$, 100 and 1000 s for Re = 20, 200 and 2000, respectively, at which each Poiseuille flow is completely steady. The corresponding CPU time is about 13 min, 2 h and 20 h on a laptop without any parallelization. Note that the function values at the Gaussian nodes are interpolated by the Interp II method for stability, thus leading to such high computational costs. Moreover, the characteristic quantities are chosen as: the length $l_c = 0.01$ m, the velocity $v_c = 0.01$ m/s, the density $\rho_c = 10^3$ kg/m^3, respectively

Fig. 5 shows the comparisons of the velocity profile between the present numerical results and theoretical solutions [39]. They are in good agreements for each Reynolds number, and all the L_1 errors are close to 1.6856×10^{-2} at the steady state. Taking Re = 2000 for example, the L_1 error is 1.1189×10^{-3}, 2.3783×10^{-3}, 4.5976×10^{-3}, 1.0×10^{-2} and 1.6856×10^{-2} at t = 20, 50, 100, 200 and 1000, respectively. Even if the simulation is run to $T = 2000$, the velocity profile is still steady. At such a high Re, the Poiseuille flow is calculated accurately, which shows the advantage of the present Gaussian SPH. It has been shown that standard SPH simulations fail at relatively large Reynolds number due to the instability, *e.g.* Re > 120 [40,41]. To further evaluate the computational reliability and accuracy of the present Gaussian SPH at large time steps, large spatial steps and small smoothing lengths, we

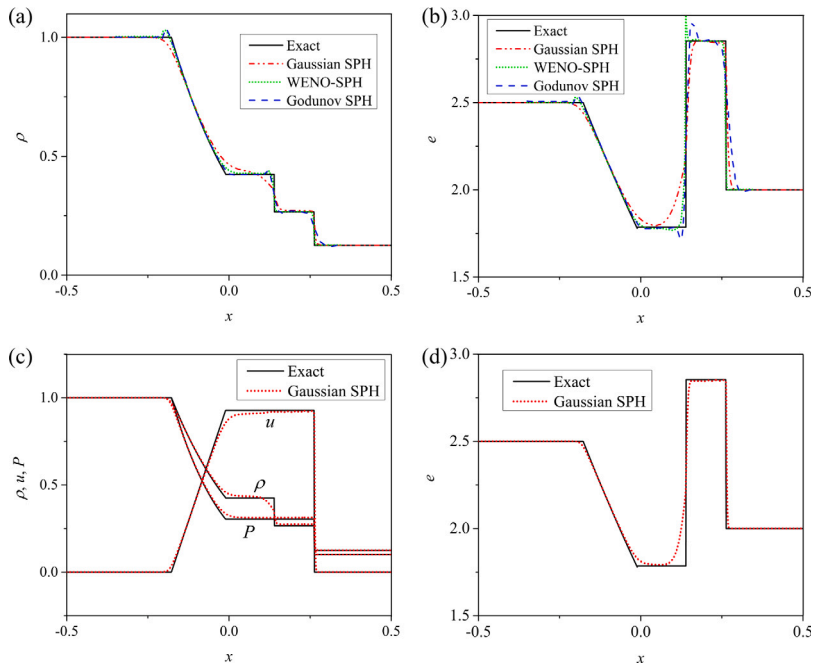


Fig. 4. Comparisons among the present Gaussian SPH, exact solutions, WENO-SPH [23] and Godunov SPH [22] in a Sod problem, (a) the density ρ and (b) the specific internal energy e under 400 SPH particles, (c) the density ρ, velocity u, pressure P and (d) the specific internal energy e under 1000 SPH particles.

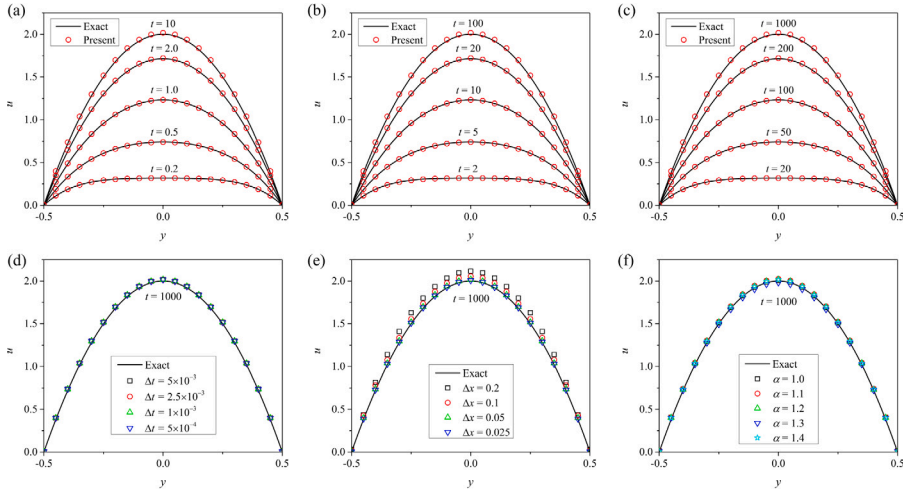


Fig. 5. Comparisons of the velocity profile between the present results and theoretical solutions in the Poiseuille flow, (a) Re = 20, (b) Re = 200, (c) Re = 2000, (d) the effect of Δt with Re = 2000, (e) the effect of Δx with Re = 2000, and (f) the effect of $h = \alpha\Delta x$ with Re = 2000. Note that the coordinate y, velocity u, time t and time step Δt are scaled by the corresponding characteristic quantities, $l_c = 0.01$ m, $v_c = 0.01$ m/s, and $t_c = 1$ s, respectively.

change the values of Δt, Δx and h one by one. The time step has no effect when it changes from 5×10^{-3} to 5×10^{-4}, but the simulation fails if $\Delta t > 5 \times 10^{-3}$. The L_1 error of the velocity profile is almost unchanged at the different time steps, as listed in Table 5, meaning that the third-order RK algorithm has a very good performance on the temporal discretization. Note that the time step is required to be less than $0.25h/(v_m + c) \approx 10^{-3}$ (here $c = 10, v_m = 2, h = 0.05$) for most SPH methods that are used to simulate the present case. The spatial step has an important effect on the accuracy of Gaussian SPH. At $\Delta x = 0.2$, there are only 50 fluid particles, and the velocity has the L_1 error of 0.10617. As Δx is decreased, the velocity error also decreases. At $\Delta x = 0.025$, there are 3200 fluid particles, and the velocity error reduces to 8.4325×10^{-3}. If $\Delta x > 0.2$, the simulation fails, but if $\Delta x < 0.025$ the computational costs are too high. At $\Delta x = 0.2$, the CPU time is about 10 min for a steady state, while it is about 72 h at $\Delta x = 0.025$. The smoothing length has a slight effect on the accuracy of Gaussian SPH. The velocity error is not monotonous with respect to h, and at $h = 1.3\Delta x$ it has a minimum value of 1.1020×10^{-2}. At $h < \Delta x$, the simulation is not correct because the ghost boundary particles are Δx away from the fluid particles at the initial state. If $h \geq 1.5\Delta x$, the simulation fails due to the instability from a large error.

3.4. Weakly compressible flow: Couette flow

Another benchmark case is the Couette flow in the space between two plates, one of which is moving tangentially relative to the other. The simulation domain is given by $L \times D = 0.02 \times 0.01$ m^2, the fluid density is $\rho = 10^3$ kg/m^3, and the relative speed of the upper plate is $v_w = 0.02$ m/s. The fluid kinematic viscosity is changed from $\nu = 10^{-5}$, 10^{-6} to 10^{-7} m^2/s, to get the different Reynolds number of $\mathrm{Re} = v_w D/\nu = 20$, 200 and 2000. There are 800 particles with $\Delta x = 0.05$, the time step is $\Delta t = 10^{-3}$, and the smoothing length is $h = \Delta x$. The

Table 5
The L_1 errors of the velocity at the steady state in the Poiseuille flow under different Δt, Δx and h, respectively.

Δt	5×10^{-3}	2.5×10^{-3}	1.0×10^{-3}	5×10^{-4}	
L_1 error	1.6856×10^{-2}	1.6856×10^{-2}	1.6856×10^{-2}	1.6856×10^{-2}	
Δx	0.2	0.2	0.05	0.025	
L_1 error	1.0617×10^{-1}	5.4169×10^{-2}	1.6856×10^{-2}	8.4325×10^{-3}	
α	1.0	1.1	1.2	1.3	1.4
L_1 error	1.6856×10^{-2}	2.4194×10^{-2}	1.2324×10^{-2}	1.1020×10^{-2}	1.6133×10^{-2}

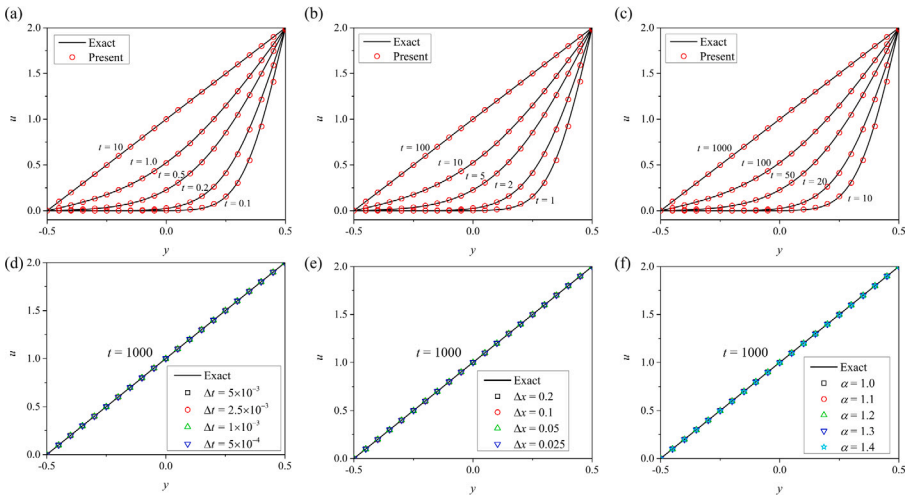


Fig. 6. Comparisons of the velocity profile between the present results and theoretical solutions in the Couette flow, (a) Re = 20, (b) Re = 200, (c) Re = 2000, (d) the effect of Δt with Re = 2000, (e) the effect of Δx with Re = 2000, and (f) the effect of $h = \alpha\Delta x$ with Re = 2000. Note that the coordinate y, velocity u, time t and time step Δt are scaled by the corresponding characteristic quantities, $l_c = 0.01$ m, $v_c = 0.01$ m/s, and $t_c = 1$ s, respectively.

Table 6
The L_1 errors of the velocity at the steady state in the Couette flow under different Δt, Δx and h, respectively.

Δt	5×10^{-3}	2.5×10^{-3}	1.0×10^{-3}	5×10^{-4}	
L_1 error	2.2294×10^{-4}	2.2295×10^{-4}	2.2295×10^{-4}	2.2295×10^{-4}	
Δx	0.2	0.1	0.05	0.025	
L_1 error	8.1900×10^{-4}	4.2057×10^{-4}	2.2295×10^{-4}	1.1555×10^{-4}	
α	1.0	1.1	1.2	1.3	1.4
L_1 error	2.2295×10^{-4}	6.4816×10^{-4}	6.9624×10^{-4}	1.5535×10^{-3}	2.3450×10^{-3}

total physical time is set to be T = 10, 100 and 1000 for Re = 20, 200 and 2000 for reaching a steady state, and the corresponding CPU time is about 10 min, 2 h and 21.5 h, respectively.

Fig. 6 shows the comparisons of the velocity profile between the present numerical results and theoretical solutions [42]. They are in good agreements for each Reynolds number, and all the L_1 errors are finally close to about 2.229×10^{-4} at the steady state. Taking Re = 2000 for example, the L_1 error decreases from 7.1956×10^{-3}, 4.9893×10^{-3}, 2.8599×10^{-3}, 1.9339×10^{-3} to 2.2295×10^{-4} when the time increases from t = 10, 20, 50, 100 to 1000. We examine the effect of three simulation parameters Δt, Δx and h. The velocity profiles at the steady state are shown in Fig. 6, and the corresponding L_1 errors are listed in Table 6. Similar to the Poiseuille flow, the time step has a negligible effect, while the spatial step and the smoothing length have important effects. As Δt is decreased from 5×10^{-3} to 5×10^{-4}, the L_1 error of velocity is almost unchanged and about 2.2295×10^{-4}. As Δx is decreased from 0.2 to 0.025 (the particle number is increased from 50 to 3200), the L_1 error decreases from 8.18×10^{-4} to 1.1555×10^{-4}, and the computational costs increase by more than 400 times. At Δx = 0.2, a very large spatial step for most SPH methods, our Gaussian SPH still performs well with high accuracy and efficiency, which demonstrates the superiority of high-order schemes. Besides, as α is increased from 1.0 to 1.4, the L_1 error monotonically increases from 2.2295×10^{-4} to 2.345×10^{-3}. It is found that the optimal smoothing length may be different for the different simulation cases. In the Poiseuille flow it is $h = 1.3\Delta x$, but $h = 1.0\Delta x$ in the Couette flow.

3.5. Weakly compressible flow: cavity flow

The lid-driven cavity is also a well-known benchmark problem for viscous incompressible flow, where the flow within a cavity is driven by a lid and a spiral flow pattern gradually develops. The simulation domain is given by $L\times D = 0.01\times0.01$ m^2, the fluid density is $\rho = 10^3$ kg/m^3, and the moving velocity of the lid is $v_w = 0.01$ m/s. The fluid kinematic viscosity is set to be $\nu = 10^{-4}$, 2.5×10^{-7} and 10^{-7} m^2/s, to get Re = 1, 400 and 1000, respectively. There are 1600 fluid particles with $\Delta x = 0.025$, $\Delta t = 10^{-4}$ and $h = \Delta x$. The total physical time is set to be T = 1, 50 and 100 for Re = 1, 400 and 1000 for reaching a steady state, and the corresponding CPU time is about 5 h, 25 h and 53 h, respectively.

Fig. 7 shows the comparisons of the velocity profiles among our results and the previously-published results. At Re = 1, the x- and y-component velocities are close to the previous results obtained by finite difference method (FDM) [32], finite volume method (FVM) [43], weakly compressible SPH (WCSPH) and incompressible SPH (ISPH) [44]. At Re = 400, the velocity profiles are much better than the WCSPH results, and close to the ISPH and FVM results. At Re = 1000, the velocity profiles also align well with the ISPH results but show slight differences compared to the FVM results. It is concluded that the WCSPH is reliable only at small Re, while the present Gaussian SPH and ISPH are still accurate at high Re. Taking Re = 400 for example, we examine the effects of simulation parameters Δt, Δx and h. The time step has almost no effect, because the numerical results are not improved

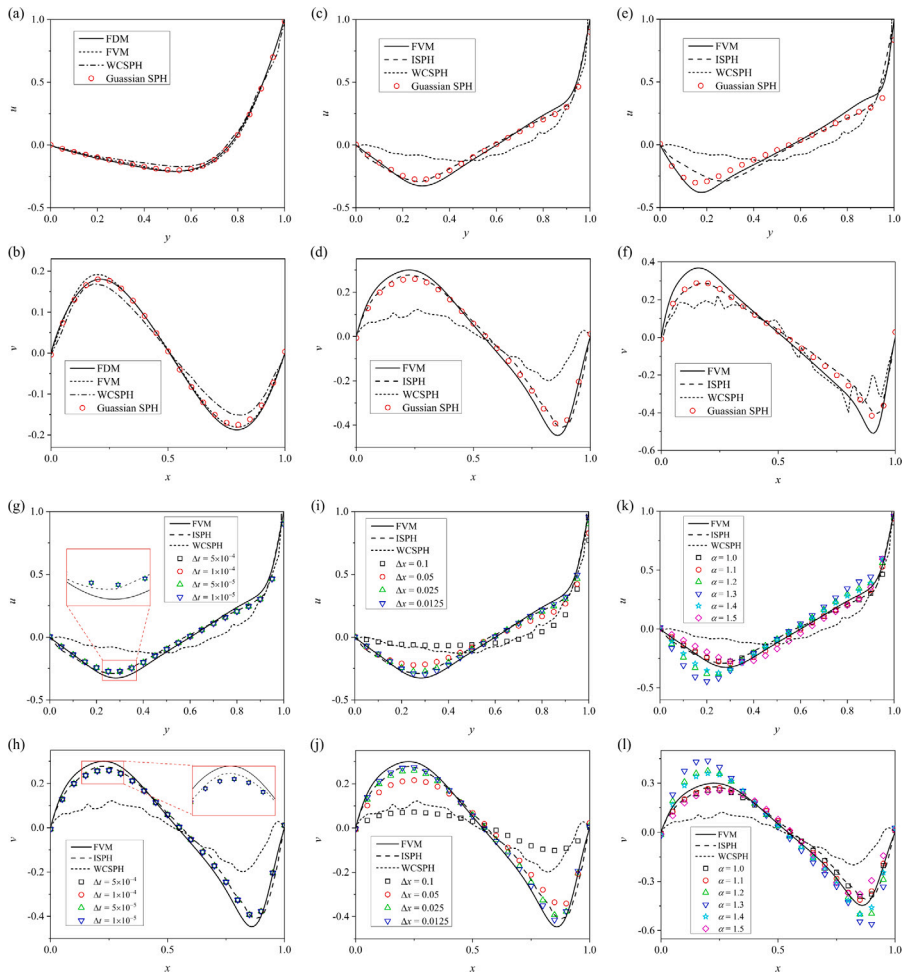


Fig. 7. Comparisons of the velocity profiles between the present results and other previously-published results in the cavity flow, (a–b) Re = 1, (c–d) Re = 400, (e–f) Re = 1000, (g–h) the effect Δt with Re = 400, (i–j) the effect of Δx with Re = 400, and (k–l) the effect of $h = \alpha\Delta x$ with Re = 400. The first and third rows are the x-component velocity u, while the second and last rows are the y-component velocity v. Note that the characteristic quantities are chosen as $l_c = 0.01$ m, $v_c = 0.01$ m/s and $t_c = 1.0$ s, respectively.

any more when Δt is decreased from 5×10^{-4} to 1×10^{-5}. At $\Delta t > 5\times10^{-4}$, the simulation fails due to the pressure instability. The spatial step still has a great effect. For example, the L_1 error decreases from 0.1310 to 0.0190 for the x-component velocity and 0.1388 to 0.0159 for the y-component velocity, as Δx is decreased from 0.1 to 0.0125, compared with the FDM results. At $\Delta x > 0.1$ the simulation fails because of the very small number of particles, while at $\Delta x < 0.0125$ it will spend more computational costs. The smooth length also has a large effect, and for example, the L_1 errors of the x-component velocity are 0.0185, 0.0250, 0.0497, 0.0789, 0.0451 and 0.0471 for α = 1.0, 1.1, 1.2, 1.3, 1.4 and 1.5, respectively.

3.6. Weakly compressible flow: Taylor–Green vortex

The Taylor–Green vortex problem is another classical problem, where several periodic vortices are formed in a square box by exerting an initial velocity field. It is a particularly challenging problem for SPH since the particles move along the streamlines towards a stagnation point leading to particle disorder [45]. The simulation domain is given by $L \times D = 0.01 \times 0.01$ m^2, and the fluid density is $\rho = 10^3$ kg/m^3. The periodic boundary conditions, instead of the wall boundary conditions, are applied to all the sides of the computational domain. Its exact solution is given by [45],

$$u(x, y, t) = -U e^{bt} \cos(2\pi x) \sin(2\pi y) \tag{93}$$

$$v(x, y, t) = U e^{bt} \sin(2\pi x) \cos(2\pi y), \tag{94}$$

where U is the characteristic velocity, $b = -8\pi^2/\mathrm{Re}$ with the Reynolds number $\mathrm{Re} = UL/\nu$. At $t = 0$ s, Eqs. (93) and (94) give the initial velocity field, and it approaches zero as t goes to infinity due to the energy dissipation. In the simulation, U is chosen as 0.01 m/s, and the fluid kinematic viscosity is $\nu = 5\times10^{-6}$ and 5×10^{-7} m^2/s, to get Re = 20 and 200. There are 1600 fluid particles with $\Delta x = 0.025$, $\Delta t = 10^{-4}$ and $h = \Delta x$.

Fig. 8 shows the comparisons of particle distribution and velocity profiles between the present results and theoretical solutions. The particle distribution is found to be consistent with the theoretical results at $t \le 0.2$, while a slight difference is observed at $t = 0.4$. Both the x- and y-component velocities are well close to the theoretical solutions, and the L_1 error of velocity is calculated to be 0.0045, 0.0025, 0.0034 and 0.0089 at t = 0.05, 0.1, 0.2 and 0.4, respectively. The kinetic energy is compared with the exact result to further assess the accuracy of the unsteady-state results, which is defined by

$$K(t) = \frac{1}{2}\sum_a m_a |\boldsymbol{v}_a|^2. \tag{95}$$

It approaches zero as t goes to infinity, and is close to the theoretical kinetic energy. We also simulate the Taylor–Green vortex at a high Reynolds number of Re = 200, and examine the effects of Δt, Δx and h, as shown in Fig. 9. Similar to all the above benchmark problems, the time step has almost no effect with the L_1 error of velocity about 0.0212. The spatial step has a large effect. At $\Delta x = 0.1$, the Gaussian SPH is still available, but has a large L_1 error of 0.0344. At $\Delta x = 0.05$,

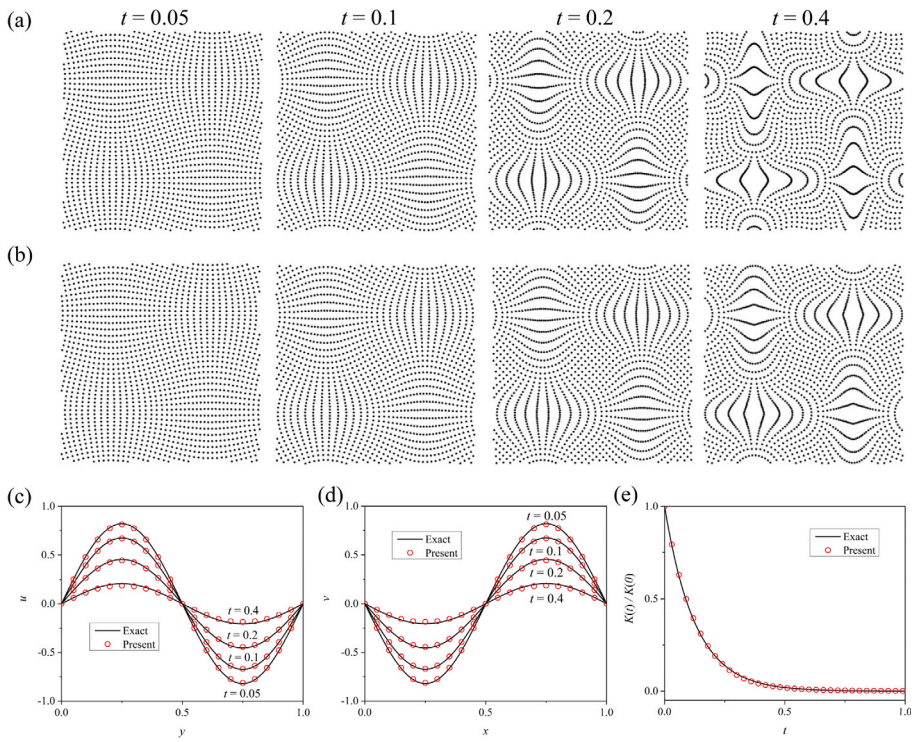


Fig. 8. Comparisons between the present results and theoretical solutions in the Taylor–Green vortex problem with Re = 20, (a) the particle distribution given by the theoretical formula, (b) the particle distribution calculated by the present Gaussian SPH, (c) the x-component velocity profile, (d) the y-component velocity profile, and (e) the kinetic energy. Note that the characteristic quantities are chosen as $l_c = 0.01$ m, $v_c = 0.01$ m/s and $t_c = 1.0$ s, respectively.

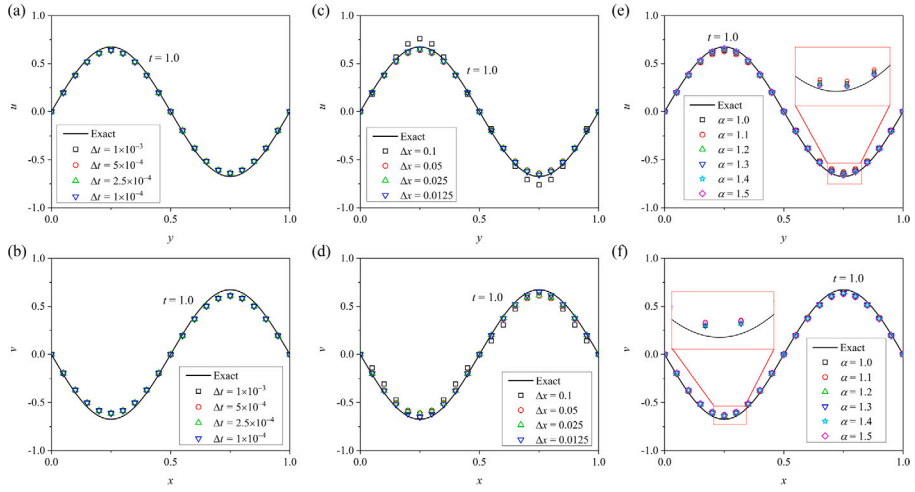


Fig. 9. The effects of (a–b) the time step Δt, (c–d) the spatial step Δx and (e–f) the smoothing length $h = \alpha\Delta x$ in the Taylor–Green vortex problem with Re = 200. The first row is the x-component velocity u, while the second row is the y-component velocity v.

the L_1 error decreases to 0.0213, and at $\Delta x \leq 0.025$, the error is close to 0.01 and is not decreased significantly, but the computational costs increase largely. The smoothing length also presents a non-monotonous trend as α is increased. A optimal length is about $h = 1.2\Delta x$ with the smallest L_1 error of 0.021, but a maximum error of 0.0268 at $h = 1.5\Delta x$.

3.7. Weakly compressible flow: dam break

The dam break flow is a complicated benchmark problem involving a free surface, liquid impact and sloshing. The simulation domain is given by $L \times D = 0.4 \times 0.4$ m^2, the fluid is assumed to be inviscid and its density is $\rho = 10^3$ kg/m^3. Initially, a rectangular fluid column in the hydrostatic equilibrium is placed between two walls in the simulation domain, and its size is $L_f \times D_f = 0.1 \times 0.2$ m^2. There are 3200 fluid particles with $\Delta x = 0.025$, $\Delta t = 10^{-4}$ and $h = 1.5\Delta x$, where the characteristic quantities are chosen as: the length $l_c = 0.1$ m, the density $\rho_c = 10^3$ kg/m^3, the gravity $g_c = 9.8$ m/s^2 and $t_c = \sqrt{g_c l_c} = 0.9899$ s. A different EOS with Eq. (3) is adopted,

$$P = P_0 \left[\left(\frac{\rho}{\rho_0} \right)^{\gamma} - 1 \right], \tag{96}$$

where $P_0 = \rho_0 c_0^2/\gamma$ with $\gamma = 7$, and ρ_0 is a reference density. This EOS is more accurate to model the fluid of water, due to the smaller density variation. Moreover, the pressure gradient is also different with Eq. (70), given by

$$\frac{\nabla P_a}{\rho_a} \approx \frac{1}{2\rho_a a_{12}^{xx}} \left[(4+d) G_1(\bar{P}\bar{\mathbf{x}}) - G_2(\bar{P}\bar{\mathbf{x}}) \right], \tag{97}$$

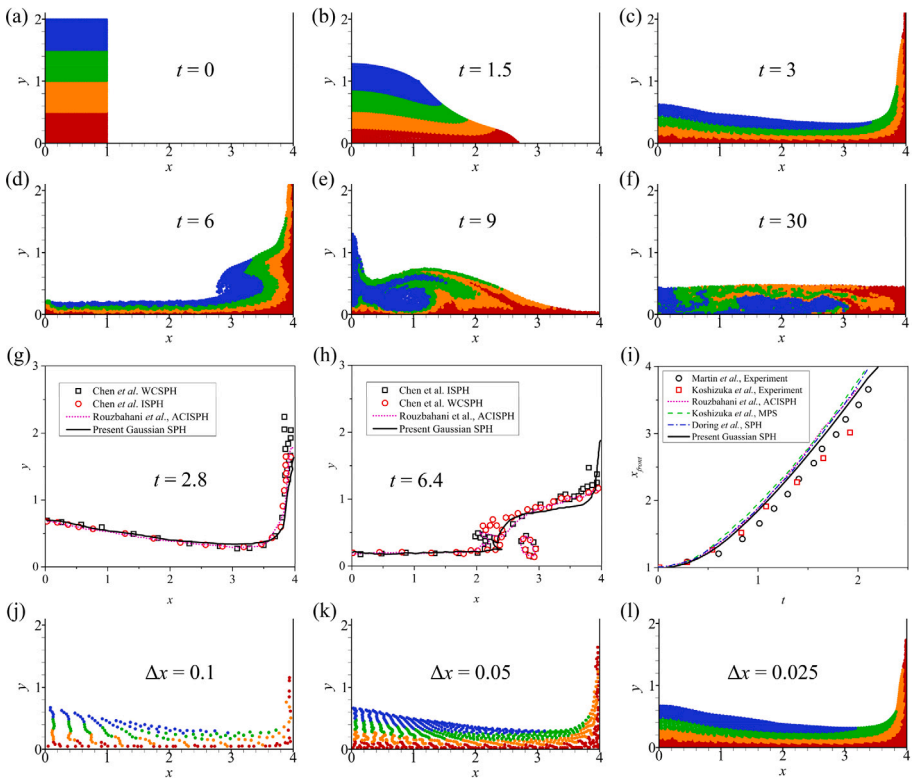


Fig. 10. Simulation results of dam break flow, (a–f) the particle distribution at the different time instants, where the particles are colored according to their initial depth to show their mutual distribution, (g–h) comparisons of the free surfaces at $t = 2.8$ and 6.4 with those obtained by WCSPH [46], ISPH [46] and ACISPH [47], (i) comparisons of the positions of water front with those obtained in the experiments [48,49] and other particle-based methods [47,49,50], (j–l) the particle distribution at $t = 2.8$ under the different spatial steps.

which gives a more accurate pressure gradient in the presence of a free surface or two phases.

Fig. 10 shows the evolution in the water domain geometry with time elapsing from the dam-breaking instant $t = 0$. The water front flows along the bottom wall at $t = 1.5$, and then it hits the right wall and runs up along the wall at $t = 3$. Once it reaches a certain height, it is reversed downward along the wall due to gravity. Then the water front plugs itself onto the free surface, and a cavity under the jet is formed at $t = 6$. After that, the water flows backward and hits the left wall at $t = 9$, and begins sloshing until a steady state is reached around $t = 30$. The water particles are distributed regularly layer by layer in the early stage, and from $t = 15$ they start mixing together. We further compare the free surfaces at $t = 2.8$ and 6.4 with those obtained by WCSPH [46], ISPH [46], and ACISPH (artificial compressibility-based incompressible SPH) [47], and also compare the position of the water front with those obtained in the experiments [48,49], MPS (moving particle semi-implicit) [49] and other SPH methods [47,49,50]. In general, a good agreement is obtained among them, showing the capability of the present Gaussian SPH. After the water impacts the right wall, such as at $t = 6.4$, an obvious difference is observed that there is no cavity formed in the present simulation, and the free surface near the right wall is also different from that obtained by other methods. The main reason is due to the boundary condition applied. In the present work, the free-slip boundary condition is implemented by adding the ghost particles outside the boundaries which are the mirrors of water particles. As a result, the run-up particles along the right wall are barely repelled by the wall and flow downward straightly. This phenomenon is also observed in the work of Staroszczyk [51] who used the same boundary condition. In the comparative SPH simulations, the ghost particles are pre-arranged at the beginning and their physical properties are interpolated by the water particles. The horizontal velocity component of the run-up particles is not exactly canceled out with the pre-arranged particles, resulting in splashing. Moreover, the present Gaussian SPH is still accurate for the dam break flow at a small spatial step of $\Delta x = 0.05$ (800 particles), and even at $\Delta x = 0.1$ (200 particles) it can also give a rough free surface.

4. Summary and outlook

In the present work, we proposed a high-order SPH method based on a Gaussian quadrature rule to simulate compressible and incompressible flows. In the kernel approximation, two kernel functions are used to approximate a function itself and its derivatives for obtaining a high accuracy order. In the particle approximation, a Gaussian quadrature rule is used to calculate an integral for having a high algebraic precision. However, the Gaussian quadrature requires Gaussian nodes that do not overlap with freely moving SPH particles, and thus their primitive variables are unknown. To reconstruct the variables at these Gaussian nodes, we compare four interpolating methods, showing that high-order interpolating methods are desired, because they directly determine the SPH accuracy and computational costs. In a 1D case, if a fifth-order weighted non-oscillatory scheme is chosen as an interpolating method, the second-order derivative of a function has a second-order convergence even when the particles are non-uniformly distributed. Several numerical examples are considered to test the proposed Gaussian SPH method, including a compressible Sod problem, incompressible Poiseuille flow, Couette flow, cavity flow, Taylor–Green vortex and dam break flow. The results show that the present method can accurately capture shock waves in the compressible flow, resolve the velocity profile in the incompressible flow with a very high Re number, and accurately reproduce the violent free surface in the dam break flow.

The proposed Gaussian SPH method has also several shortcomings and needs improvement in the future. First, a high-order and time-saving interpolation method is desired to reconstruct the variables at Gaussian nodes from a set of Lagrangian particles, instead of a set

of Eulerian particles. Such interpolation is still challenging in high dimensions so far. Second, a thorough theoretical analysis is required to develop its foundation, including consistency, convergence and stability. Particularly, in the present work we use two kernel functions which jointly appear as negative weights and impair the stability. Third, parallelization is necessary to meet the needs of industrial applications, especially GPU parallelization. The proposed Gaussian SPH is more accurate, compared with other SPH methods, but its computational costs are also much higher because of the interpolation at Gaussian nodes. For a simple 3D Poiseuille flow, the computational costs are about 100 times higher than the conventional SPH. More complex flow problems will be considered in the future when computational costs are reduced with new interpolation algorithms and parallelization techniques.

CRediT authorship contribution statement

Ni Sun: Writing – original draft, Validation, Methodology. **Ting Ye:** Writing – review & editing, Supervision, Methodology. **Zehong Xia:** Investigation. **Zheng Feng:** Investigation. **Rusheng Wang:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Acknowledgments

The authors gratefully acknowledge the financial support from the Ministry of Science and Technology of China under Project No. 2021YFA0719103 and from the National Natural Science Foundation of China under Project No. 12071178, and also would like to thank School of Mathematics at Jilin University of China for all the support, especially the valuable discussions with the members and the access to computational resources.

Appendix A. Kernel approximation of function derivatives

In order to conduct the kernel approximation of the first-order derivatives, we multiply Eq. (24) by two odd functions $(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)$ and $(\boldsymbol{x}-\boldsymbol{x}_a)W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)$ respectively, and integrate them over Ω,

$$\begin{aligned}\int_\Omega (f(\boldsymbol{x})-f(\boldsymbol{x}_a))g(\boldsymbol{x})(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}\\ =\sum_{k=1}^{6}\frac{1}{k!}\left(\sum_{i=1}^{k}C_k^i f^{(i)}(\boldsymbol{x}_a)g^{(k-i)}(\boldsymbol{x}_a)\right)\odot\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^{k+1}\\ \times W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}+O\left(h^8\right),\end{aligned}\tag{A.1}$$

$$\begin{aligned}\int_\Omega (f(\boldsymbol{x})-f(\boldsymbol{x}_a))g(\boldsymbol{x})(\boldsymbol{x}-\boldsymbol{x}_a)W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}\\ =\sum_{k=1}^{6}\frac{1}{k!}\left(\sum_{i=1}^{k}C_k^i f^{(i)}(\boldsymbol{x}_a)g^{(k-i)}(\boldsymbol{x}_a)\right)\odot\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^{k+1}\\ \times W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}+O\left(h^8\right),\end{aligned}\tag{A.2}$$

with noting

$$\int_\Omega W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}=1\quad\text{and}\quad\int_\Omega W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}=d,\tag{A.3}$$

where d is the number of dimensions.

Define

$$\boldsymbol{a}_{1k}=\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^k W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x},\tag{A.4}$$

$$\boldsymbol{a}_{2k}=\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^k W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x},\tag{A.5}$$

$$\boldsymbol{b}_1=\int_\Omega[f(\boldsymbol{x})-f(\boldsymbol{x}_a)]g(\boldsymbol{x})(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x},\tag{A.6}$$

$$\boldsymbol{b}_2=\int_\Omega[f(\boldsymbol{x})-f(\boldsymbol{x}_a)]g(\boldsymbol{x})(\boldsymbol{x}-\boldsymbol{x}_a)W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}.\tag{A.7}$$

It is clear that $\boldsymbol{a}_{1k}$ and $\boldsymbol{a}_{2k}$ are scalars for 1D case but kth-order tensors for others, $\boldsymbol{b}_1$ and $\boldsymbol{b}_2$ are also scalars for 1D case but first-order tensors for others. Considering the parities of $(\boldsymbol{x}-\boldsymbol{x}_a)^k W(\boldsymbol{x}-\boldsymbol{x}_a,h)$ and $(\boldsymbol{x}-\boldsymbol{x}_a)^k W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)$ gives

$$\boldsymbol{a}_{1k}=\boldsymbol{0}\quad\text{and}\quad\boldsymbol{a}_{2k}=\boldsymbol{0},\quad k=1,3,5,\tag{A.8}$$

such that Eqs. (A.1) and (A.2) are converted into

$$f'(\boldsymbol{x}_a)g(\boldsymbol{x}_a)\odot\boldsymbol{a}_{12}+\frac{1}{3!}\sum_{i=1}^{3}C_3^i f^{(i)}(\boldsymbol{x}_a)g^{(3-i)}(\boldsymbol{x}_a)\odot\boldsymbol{a}_{14}=\boldsymbol{b}_1+O\left(h^6\right),\tag{A.9}$$

$$f'(\boldsymbol{x}_a)g(\boldsymbol{x}_a)\odot\boldsymbol{a}_{22}+\frac{1}{3!}\sum_{i=1}^{3}C_3^i f^{(i)}(\boldsymbol{x}_a)g^{(3-i)}(\boldsymbol{x}_a)\odot\boldsymbol{a}_{24}=\boldsymbol{b}_2+O\left(h^6\right),\tag{A.10}$$

with truncating to the fifth-order derivative.

Now we develop the relationship among $\boldsymbol{a}_{12}$, $\boldsymbol{a}_{14}$, $\boldsymbol{a}_{22}$ and $\boldsymbol{a}_{24}$, and solve $f'(\boldsymbol{x}_a)$ from Eqs. (A.9) and (A.10). Since

$$-(\boldsymbol{x}-\boldsymbol{x}_a)\cdot W'(\boldsymbol{x}-\boldsymbol{x}_a,h)=-\nabla\cdot[(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)]+W(\boldsymbol{x}-\boldsymbol{x}_a,h)\nabla\cdot(\boldsymbol{x}-\boldsymbol{x}_a)\tag{A.11}$$

multiplying $(\boldsymbol{x}-\boldsymbol{x}_a)^k$ from both sides and integrating them over Ω yields

$$\begin{aligned}-\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^{k+1}\cdot W'(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}=&-\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^k\nabla\cdot[(\boldsymbol{x}-\boldsymbol{x}_a)\\ &\times W(\boldsymbol{x}-\boldsymbol{x}_a,h)]\mathrm{d}\boldsymbol{x}\\ &+\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^k W(\boldsymbol{x}-\boldsymbol{x}_a,h)\nabla\\ &\cdot(\boldsymbol{x}-\boldsymbol{x}_a)\mathrm{d}\boldsymbol{x}.\end{aligned}\tag{A.12}$$

Note that the left term of Eq. (A.12) is

$$-\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^{k+1}\cdot W'(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}=\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^k W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}=\boldsymbol{a}_{2k},\tag{A.13}$$

while the second term on the right side of Eq. (A.12) is

$$\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^k W(\boldsymbol{x}-\boldsymbol{x}_a,h)\nabla\cdot(\boldsymbol{x}-\boldsymbol{x}_a)\mathrm{d}\boldsymbol{x}=d\int_\Omega(\boldsymbol{x}-\boldsymbol{x}_a)^k W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}=d\boldsymbol{a}_{1k},\tag{A.14}$$

where d is the number of dimensions. Take any component of $(\boldsymbol{x}-\boldsymbol{x}_a)^k$ to handle the first term on the right side of Eq. (A.12), which is denoted as $x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k}$ with the indices of $\alpha_i=1,2,3$ for $i=1,2,\ldots,k$, and thus $x_1=x-x_a, x_2=y-y_a, x_3=z-z_a$. Since

$$\begin{aligned}-(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k})\nabla\cdot[(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)]=&-\nabla\cdot[(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k})(\boldsymbol{x}-\boldsymbol{x}_a)\\ &\times W(\boldsymbol{x}-\boldsymbol{x}_a,h)]\\ &+\nabla(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k})\cdot(\boldsymbol{x}-\boldsymbol{x}_a)\\ &\times W(\boldsymbol{x}-\boldsymbol{x}_a,h),\end{aligned}\tag{A.15}$$

integrating both sides over Ω yields

$$-\int_\Omega (x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k})\nabla\cdot[(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)]\mathrm{d}\boldsymbol{x} = -\int_\Omega \nabla\cdot[(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k}) \times (\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)]\mathrm{d}\boldsymbol{x} + \int_\Omega \nabla(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k}) \cdot(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}. \tag{A.16}$$

By using Gauss's law and $W(\boldsymbol{x}-\boldsymbol{x}_a,h)=0$ on $\partial\Omega$, the first term on the right side of Eq. (A.16) is

$$\int_\Omega \nabla\cdot[(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k})(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)]\mathrm{d}\boldsymbol{x} = \oint_{\partial\Omega}[(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k}) \times(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)]\cdot\mathrm{d}\boldsymbol{S} = 0. \tag{A.17}$$

The second term on the right side of Eq. (A.16) is

$$\begin{aligned}\int_\Omega \nabla(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k})\cdot(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x} = \int_\Omega \Big[& x_1\frac{\partial}{\partial x_1}(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k}) \times W(\boldsymbol{x}-\boldsymbol{x}_a,h) \\ & + x_2\frac{\partial}{\partial x_2}(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k}) \times W(\boldsymbol{x}-\boldsymbol{x}_a,h) \\ & + x_3\frac{\partial}{\partial x_3}(x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k}) \times W(\boldsymbol{x}-\boldsymbol{x}_a,h)\Big]\mathrm{d}\boldsymbol{x} \\ = & k\int_\Omega (x_{\alpha_1}x_{\alpha_2}\cdots x_{\alpha_k}) \times W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x},\end{aligned} \tag{A.18}$$

such that the first term on the right side of Eq. (A.12) is

$$-\int_\Omega (\boldsymbol{x}-\boldsymbol{x}_a)^k\nabla\cdot[(\boldsymbol{x}-\boldsymbol{x}_a)W(\boldsymbol{x}-\boldsymbol{x}_a,h)]\mathrm{d}\boldsymbol{x} = k\int_\Omega (\boldsymbol{x}-\boldsymbol{x}_a)^k W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x} = k\boldsymbol{a}_{1k}. \tag{A.19}$$

Combining Eqs. (A.13), (A.14) and (A.19), we obtain

$$\boldsymbol{a}_{2k} = (k+d)\boldsymbol{a}_{1k}, \tag{A.20}$$

and as $k=2$ and 4

$$\boldsymbol{a}_{22} = (2+d)\boldsymbol{a}_{12}, \quad \boldsymbol{a}_{24} = (4+d)\boldsymbol{a}_{14}. \tag{A.21}$$

Substituting them into Eqs. (A.9) and (A.10), we have

$$f'(\boldsymbol{x}_a) = \frac{1}{2g(\boldsymbol{x}_a)}\left[(4+d)\boldsymbol{b}_1-\boldsymbol{b}_2\right]\odot\boldsymbol{a}_{12}^{-1}+O\left(h^4\right), \tag{A.22}$$

Because of the symmetry and parity of $\boldsymbol{a}_{1k}$,

$$a_{12}^{xx} = a_{12}^{yy} = a_{12}^{zz} = \int_\Omega (x-x_a)^2 W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{A.23}$$

$$a_{12}^{xy} = a_{12}^{yx} = a_{12}^{xz} = a_{12}^{zx} = a_{12}^{yz} = a_{12}^{zy} = 0, \tag{A.24}$$

where $a_{12}^{\alpha\beta}$ is the $\alpha\beta$-component of $\boldsymbol{a}_{12}$ with $\alpha,\beta = x,y,z$. Therefore, Eq. (A.22) is simplified into Eq. (25), which is the final form of the kernel approximation for the first-order derivative.

For the kernel approximation of the second-order derivatives, we multiply Eq. (21) by two even functions $W(\boldsymbol{x}-\boldsymbol{x}_a,h)$ and $W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)$ respectively, and integrating them over Ω,

$$\int_\Omega f(\boldsymbol{x})W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x} = \sum_{k=0}^{6}\frac{f^{(k)}(\boldsymbol{x}_a)}{k!}\odot\int_\Omega (\boldsymbol{x}-\boldsymbol{x}_a)^k W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x} + O\left(h^7\right), \tag{A.25}$$

$$\int_\Omega f(\boldsymbol{x})W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x} = \sum_{k=0}^{6}\frac{f^{(k)}(\boldsymbol{x}_a)}{k!}\odot\int_\Omega (\boldsymbol{x}-\boldsymbol{x}_a)^k W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x} + O\left(h^7\right). \tag{A.26}$$

By using Eq. (A.8), Eqs. (A.25) and (A.26) are converted into

$$\frac{1}{2!}f''(\boldsymbol{x}_a)\odot\boldsymbol{a}_{12}+\frac{1}{4!}f^{(4)}(\boldsymbol{x}_a)\odot\boldsymbol{a}_{14} = c_1+O\left(h^6\right), \tag{A.27}$$

$$\frac{1}{2!}f''(\boldsymbol{x}_a)\odot\boldsymbol{a}_{22}+\frac{1}{4!}f^{(4)}(\boldsymbol{x}_a)\odot\boldsymbol{a}_{24} = c_2+O\left(h^6\right), \tag{A.28}$$

where

$$c_1 = \int_\Omega [f(\boldsymbol{x})-f(\boldsymbol{x}_a)]W(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}, \tag{A.29}$$

$$c_2 = \int_\Omega [f(\boldsymbol{x})-f(\boldsymbol{x}_a)]W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)\mathrm{d}\boldsymbol{x}. \tag{A.30}$$

Applying the identities in Eq. (A.21) yields Eq. (39), which is the final form of kernel approximation for the second-order derivative.

Therefore, the kernel approximation for the first- and second-order derivatives in the Gaussian SPH is given for a 1D case by,

$$\begin{aligned}\nabla f(\boldsymbol{x}_a) &= \frac{1}{2g(\boldsymbol{x}_a)}\frac{5\boldsymbol{b}_1-\boldsymbol{b}_2}{a_{12}^{xx}}+O\left(h^4\right), \\ \nabla^2 f(\boldsymbol{x}_a) &= f''_{\alpha\alpha}(\boldsymbol{x}_a) = \frac{5c_1-c_2}{a_{12}^{xx}}+O\left(h^4\right),\end{aligned} \tag{A.31}$$

for a 2D case,

$$\begin{aligned}\nabla f(\boldsymbol{x}_a) &= \frac{1}{2g(\boldsymbol{x}_a)}\frac{6\boldsymbol{b}_1-\boldsymbol{b}_2}{a_{12}^{xx}}+O\left(h^4\right), \\ \nabla^2 f(\boldsymbol{x}_a) &= f''_{\alpha\alpha}(\boldsymbol{x}_a) = \frac{6c_1-c_2}{a_{12}^{xx}}+O\left(h^4\right),\end{aligned} \tag{A.32}$$

and for a 3D case,

$$\begin{aligned}\nabla f(\boldsymbol{x}_a) &= \frac{1}{2g(\boldsymbol{x}_a)}\frac{7\boldsymbol{b}_1-\boldsymbol{b}_2}{a_{12}^{xx}}+O\left(h^4\right), \\ \nabla^2 f(\boldsymbol{x}_a) &= f''_{\alpha\alpha}(\boldsymbol{x}_a) = \frac{7c_1-c_2}{a_{12}^{xx}}+O\left(h^4\right).\end{aligned} \tag{A.33}$$

Appendix B. Second-type Gaussian quadrature operator

In order to numerically calculate the second integral in Eq. (48), the second-type Gaussian quadrature operator is introduced, which is similar to the first-type Gaussian quadrature operator in Eq. (56). Take the kernel function $W_1(\boldsymbol{x}-\boldsymbol{x}_a,h)$ as the weighted function, and applying Eq. (29) yields

$$\begin{aligned}W_1(\boldsymbol{x}-\boldsymbol{x}_a,h) = & -(x-x_a)W'(x-x_a,h)W(y-y_a,h)W(z-z_a,h) \\ & -(y-y_a)W(x-x_a,h)W'(y-y_a,h)W(z-z_a,h) \\ & -(z-z_a)W(x-x_a,h)W(y-y_a,h)W'(z-z_a,h),\end{aligned} \tag{B.1}$$

such that

$$\begin{aligned}I_2(f) = & \int_\Omega f(\boldsymbol{x})\left[-(x-x_a)W'(x-x_a,h)W(y-y_a,h)W(z-z_a,h)\right]\mathrm{d}\boldsymbol{x} \\ & + \int_\Omega f(\boldsymbol{x})\left[-(y-y_a)W(x-x_a,h)W'(y-y_a,h)W(z-z_a,h)\right]\mathrm{d}\boldsymbol{x} \\ & + \int_\Omega f(\boldsymbol{x})\left[-(z-z_a)W(x-x_a,h)W(y-y_a,h)W'(z-z_a,h)\right]\mathrm{d}\boldsymbol{x}.\end{aligned} \tag{B.2}$$

Considering the Gaussian quadrature on one dimension, we have

$$\int_{x_a-2h}^{x_a+2h} f(x)W_1(x-x_a,h)\mathrm{d}x = \sum_{i=1}^{3}A'_i f(x_a+2hr'_i)+O\left(h^6\right), \tag{B.3}$$

where the quadrature coefficients and Gaussian nodes are

$$A'_1 = A'_2 = A'_3 = \frac{1}{3}, \quad r'_1 = -\frac{\sqrt{6}}{4}, \ r'_2 = 0, \ r'_3 = \frac{\sqrt{6}}{4}. \tag{B.4}$$

Applying the product formula of multiple integral again, we obtain from Eq. (B.2)

$$I_2(f) = \sum_{i,j,k=1}^{3} A_i' A_j A_k f(x_a + 2hr_i', y_a + 2hs_j, z_a + 2ht_k) + \sum_{i,j,k=1}^{3} A_i A_j' A_k f(x_a + 2hr_i, y_a + 2hs_j', z_a + 2ht_k) + \sum_{i,j,k=1}^{3} A_i A_j A_k' f(x_a + 2hr_i, y_a + 2hs_j, z_a + 2ht_k') + O\left(h^6\right). \tag{B.5}$$

By combining the quadrature coefficients together, we obtain the second-type Gaussian quadrature operator $G_2(f)$ in Eq. (57), and the second integral in Eq. (48) is finally given by

$$I_2(f) = G_2(f) + O\left(h^6\right). \tag{B.6}$$

References

[1] Gingold RA, Monaghan JJ. Smoothed particle hydrodynamics: theory and application to non-spherical stars. Mon Not R Astron Soc 1977;181:375–89.
[2] Lucy LB. A numerical approach to testing of fission hypothesis. Astron J 1997;82:1013–24.
[3] Takeda H, Miyama SM, Sekiya M. Numerical simulation of viscous flow by smoothed particle hydrodynamics. Progr Theoret Phys 1994;92:939–60.
[4] Kum O, Hoover WG, Posch HA. Viscous conducting flows with smooth-particle applied mechanics. Phys Rev E 1995;52:4899.
[5] Cleary PW, Monaghan JJ. Conduction modelling using smoothed particle hydrodynamics. J Comput Phys 1999;148:227–64.
[6] Español P, Revenga M. Smoothed dissipative particle dynamics. Phys Rev E 2003;67:026705.
[7] Shadloo MS, Oger G, Le Touzé D. Smoothed particle hydrodynamics method for fluid flows towards, industrial applications: motivations, current state, and challenges. Comput Fluids 2016;136:11–34.
[8] Xu X, Yu P. A multiscale sph method for simulating transient viscoelastic flows using bead–spring chain model. J Non-Newton Fluid Mech 2016;229:27–42.
[9] Xu X, Tian L, Peng S, Yu P. Development of sph for simulation of non-isothermal viscoelastic free surface flows with application to injection molding. Appl Math Model 2022;104:782–805.
[10] Xu X, Tian L, Yu P. Multiscale sph simulations of viscoelastic injection molding processes based on bead–spring chain model. Eng Anal Bound Elem 2023;149:213–30.
[11] Violeau D, Rogers BD. Smoothed particle hydrodynamics (SPH) for free-surface flows: past, present and future. J Hydraul Res 2016;54:1–26.
[12] Zhang AM, Sun PN, Ming FR, Colagrossi A. Smoothed particle hydrodynamics and its applications in fluid–structure interactions. J Hydrodyn 2017;29:187–216.
[13] Ye T, Pan DY, Huang C, Liu MB. Smoothed particle hydrodynamics (SPH) for complex fluid flows: Recent developments in methodology and applications. Phys Fluids 2019;31:011301.
[14] Lind SJ, Rogers BD, Stansby PK. Review of smoothed particle hydrodynamics: towards converged lagrangian flow modelling. Proc R Soc A 2020;476:20190801.
[15] Khayyer A, Gotoh H, Shimizu Y. Comparative study on accuracy and conservation properties of two particle regularization schemes and proposal of an optimized particle shifting scheme in isph context. J Comput Phys 2017;332:236–56.
[16] Peng YX, Zhang AM, Ming FR. Particle regeneration technique for smoothed particle hydrodynamics in simulation of compressible multiphase flows. Comput Methods Appl Mech Engrg 2021;376:113653.
[17] Randles PW, Libersky LD. Smoothed particle hydrodynamics: Some recent improvements and applications. Comput Methods Appl Mech Engrg 1996;139:375–408.
[18] Chen J, Beraun J. A generalized smoothed particle hydrodynamics method for nonlinear dynamic problems. Comput Methods Appl Mech Engrg 2000;190:225–39.
[19] Liu M, Xie W, Liu G. Modeling incompressible flows using a finite particle method. Appl Math Model 2005;29:1252–70.
[20] Zhang Z, Liu M. A decoupled finite particle method for modeling incompressible flows with free surfaces. Appl Math Model 2018;60:606–33.
[21] Zhang ZL, Walayat K, Chang JZ, Liu MB. Meshfree modeling of a fluid-particle two-phase flow with an improved sph method. Internat J Numer Methods Engrg 2018;116:530–69.
[22] Puri K, Ramachandran P. A comparison of SPH schemes for the compressible Euler equations. J Comput Phys 2014;256:308–33.
[23] Wang PP, Zhang AM, Meng ZF, Ming FR, Fang XL. A new type of WENO scheme in SPH for compressible flows with discontinuities. Comput Methods Appl Mech Engrg 2021;381:113770.
[24] Lind SJ, Xu R, Stansby PK, Rogers BD. Incompressible smoothed particle hydrodynamics for free-surface flows: A generalised diffusion-based algorithm for stability and validations for impulsive flows and propagating waves. J Comput Phys 2012;231:1499–523.
[25] Sun PN, Colagrossi A, Marrone S, Antuono M, Zhang AM. A consistent approach to particle shifting in the δ-Plus-SPH model. Comput Methods Appl Mech Engrg 2019;348:912–34.
[26] Gao T, Fu L. A new particle shifting technique for sph methods based on voronoi diagram and volume compensation. Comput Methods Appl Mech Engrg 2023;404:0045–7825.
[27] Kitsionas S, Whitworth AP. Smobothed particle hydrodynamics with particle splitting, applied to self-gravitating collapse. Mon Not R Astron Soc 2002;330:129–36.
[28] Xiong Q, Li B, Xu J. GPU-accelerated adaptive particle splitting and merging in SPH. Comput Phys Comm 2013;184:1701–7.
[29] Yang XF, Kong SC, Liu MB, Liu QQ. Smoothed particle hydrodynamics with adaptive spatial resolution (SPH-ASR) for free surface flows. J Comput Phys 2021;443:110539.
[30] Sun PN, Colagrossi A, Marrone S, Zhang AM. The δ-Plus-SPH model: simple procedures for a further improvement of the sph scheme. Comput Methods Appl Mech Engrg 2017;315:25–49.
[31] Antuono M, Colagrossi A, Marrone S, Molteni D. Free-surface flows solved by means of sph schemes with numerical diffusive terms. Comput Phys Comm 2010;181:532–49.
[32] Liu GR, Liu MB. Smoothed particle hydrodynamics: a meshfree particle method. World Scientific; 2003.
[33] Quinlan NJ, Basa M, Lastiwka M. Truncation error in mesh-free particle methods. Internat J Numer Methods Engrg 2006;66:2064–85.
[34] Zhu Q, Hernquist L, Li Y. Numerical convergence in smoothed particle hydrodynamics. Astrophys J 2015;800:6.
[35] Bui HH, Nguyen GD. Smoothed particle hydrodynamics (SPH) and its applications in geomechanics: From solid fracture to granular behaviour and multiphase flows in porous media. Comput Geotech 2021;138:104315.
[36] Hammer PC, Wymore AW. Numerical evaluation of multiple integrals i. Math Tables Other Aids Comput 1957;11:59–67.
[37] Shu CW. Essentially non-oscillatory and weighted essentially non-oscillatory schemes. Acta Numer 2020;29:701–62.
[38] Sod GA. A survey of several finite difference methods for systems of nonlinear hyperbolic conservation laws. J Comput Phys 1978;27:1–31.
[39] Sigalotti LDG, Klapp J, Sira E, Meleán Y, Hasmy A. SPH simulations of time-dependent Poiseuille flow at low Reynolds numbers. J Comput Phys 2003;191:622–38.
[40] Meister M, Burger G, Rauch W. On the Reynolds number sensitivity of smoothed particle hydrodynamics. J Hydraul Res 2014;52:824–35.
[41] Song B, Pazouki A, Pöschel T. Instability of smoothed particle hydrodynamics applied to Poiseuille flows. Comput Math Appl 2018;76:1447–57.
[42] Morris JP, Fox PJ, Zhu Y. Modeling low Reynolds number incompressible flows using SPH. J Comput Phys 1997;136:214–26.
[43] Chiang TP, Sheu WH, Hwang RR. Effect of Reynolds number on the eddy structure in a lid-driven cavity. Internat J Numer Methods Fluids 1998;26:557–79.
[44] Lee ES, Moulinec C, Xu R, Violeau D, Laurence D, Stansby P. Comparisons of weakly compressible and truly incompressible algorithms for the SPH mesh free particle method. J Comput Phys 2008;227:8417–36.
[45] Ramachandran P, Muta A, Ramakrishna M. Dual-time smoothed particle hydrodynamics for incompressible fluid simulation. Comput Fluids 2021;227:105031.
[46] Chen Z, Zong Z, Liu MB, Li HT. A comparative study of truly incompressible and weakly compressible sph methods for free surface incompressible flows. Internat J Numer Methods Fluids 2013;73:813–29.
[47] Rouzbahani F, Hejranfar K. A truly incompressible smoothed particle hydrodynamics based on artificial compressibility method. Comput Phys Comm 2017;210:10–28.
[48] Martin JC, Moyce WJ. Part iv. an experimental study of the collapse of liquid columns on a rigid horizontal plane. Philos Trans R Soc Lond 1952;244:312–24.
[49] Koshizuka S, Oka Y. Moving particle semi-implicit method: Fully lagrangian analysis of incompressible flows. In: Proceedings of the European congress on computational methods in applied sciences and engineering. Barcelona, Spain; 2000, p. 11–4.
[50] Doring M, Andrillon Y, Alessandrini B, Ferrant P. Sph free surface flow simulation. In: Proceedings of 18th international workshop on water waves and floating bodies. Le Croisic, France; 2003.
[51] Staroszczyk R. Simulation of dam-break flow by a corrected smoothed particle hydrodynamics method. Arch Hydro Eng Environ Mech 2010;57:61–79.